

Preliminary Data Analysis (Assessment 1)

Data Mining

Student Number: 100518675

Statement of AI usage: *AI was used to identify appropriate pandas' methods for Exploratory Data Analysis (EDA), display functions for familiarization, highlight potential high/low level tasks, and guide the overall structure of the workflow and report.*

1. Executive Summary

This exploratory data analysis (EDA) explores the provided supermarket dataset, which is a survey of shoppers across five store locations (Wymondham, Fakenham, Norwich, Dereham, and Thetford), capturing demographic, behavioural, and experiential information for each visit. The dataset contains 5,000 rows across 23 columns, including customer demographics, shopping behaviours, basket composition, payment method, and satisfaction levels, alongside optional free-text customer comments. It provides a broad picture of who the customers are, how they shop, and how they feel about their experience. The learning task implemented in this assignment includes regression, predicting a customer's satisfaction score from their demographics and shopping behaviour. The target variable, satisfaction, was encoded on an ordinal 1-5 scale to support modelling. Exploratory analysis revealed meaningful relationships between satisfaction and several predictors, supporting the feasibility of this task. The primary data quality concerns include moderate to high missingness in customer comment (31.5%), distance to store (18.2%), and to a lesser extent, household income (3.8%) and typical payment method (4.0%). Extreme outliers were identified in age and household income, and high cardinality due to inconsistent entries was observed in store location and typical shopping mission, requiring fuzzy matching and manual correction. Key insights from the familiarisation stage indicate that the majority of customers reported medium satisfaction. Higher household income was unexpectedly associated with lower satisfaction, and shoppers aged 40-49 and 50-59 represented the largest customer segment. The dataset is sufficiently structured to support the proposed regression task, though the inclusion of additional variables, such as employee interaction and shopping frequency, would meaningfully strengthen future analysis. The report is organised into the following sections: a Preliminary Data Analysis Report, supported by appendices that include the Jupyter Notebook (analytical code), Visualisations (Appendix B-C, Data Dictionary, and Travel Log.

2. Introduction

The goal of this preliminary analysis is to examine the structure, quality, and behavioural patterns within the dataset in order to gain any insight about supermarket shoppers and assess the feasibility of a predictive learning task. Evaluation encompasses the completeness, consistency, and distribution of variables, with the aim of informing how the data might be used to enhance shopper experiences and store operations. The dataset combines demographic and behavioural information captured across five store locations. It includes detailed basket composition, satisfaction scores, and customer comments, offering a varied view of each shopping visit. While the data is mostly complete, notable missingness is present in select fields, and categorical variables exhibit inconsistencies that require attention prior to analysis.

3. Data Familiarisation

3.1 Field Descriptions

The dataset captures a range of indicators for each visit. Age and household income represent estimated annual earnings. Gender is self-reported, and store location identifies the branch visited. Typical queue wait time measures the usual time spent waiting to pay, while distance to store estimates the shopper's proximity to the store. Percent self-service checkout uses records the proportion of visits completed via self-service. The children field captures the number of dependents in the household. Basket composition is recorded through an item counts variable across six categories: fruit and veg, meat, cupboard, frozen, toiletries, and other items.

Additional engagement and preference variables include has loyalty card, typical visit time, typical shopping mission, average time in store, and typical payment method. Finally, satisfaction captures the shopper's reported experience level, and customer comment provides optional text feedback.

3.2 Structure, Statistics, and Distributions

Demographic and economic variables include age (median 46, range 20-367) and household income (median 35,960k, maximum 10.03m). Behavioural measures include typical queue wait time (median 12.0), average time in store (median 63.7), and percent self-service checkout uses (median 0.30). Categorical variables include gender (5 types, with 81 missing), typical shopping mission, typical payment method, and a customer comment field. A broadly balanced gender split was observed (female: 2,192 and male: 2,185) (**Appendix B, Figure 1**). Histograms revealed right skew across several numeric fields (**Appendix B, Figure 2**). Outlier flags identified 67 rows with age values outside the 18-120 range, and an IQR analysis of household income ($Q1 = 25,465l$ and $Q3 = 52,029k$) identified approximately 258 values exceeding the $1.5 \times IQR$ upper cap of 91,876k. Missingness was modest but elevated for customer comments (31.5%) and distance to store (18.2%), with smaller gaps in typical payment method (4.0%) and household income (3.8%). High cardinality due to inconsistent text entries was noted in the store location (162 unique strings) and typical shopping mission (40 unique strings).

4. Data Transformations

4.1 Cleaning and Preparation Steps

During the data transformation stage, a series of cleaning steps was applied to prepare the dataset for reliable analysis. Columns not relevant to the scope of the analysis, including brings own bag and brand loyalty, were removed to reduce potential unnecessary complexity. Fuzzy matching techniques were then applied to correct inconsistent spellings in store location and typical shopping mission, ensuring entries aligned with valid reference lists. Manual cleaning was performed to standardise the typical payment method, addressing common misspellings and formatting inconsistencies. Missing values were addressed by filling customer comments with empty strings, assigning an unknown label to missing gender and payment method entries, and imputing median values for key numerical fields, including household income, queue wait time, distance to store, frozen items, and toiletries items. Outliers were controlled by clipping age values outside the 18-120 range, constraining household income using IQR-based thresholds, and removing excessively high queue wait times above 1,800. Additional checks confirmed that the remaining numerical fields fell within reasonable ranges. Data types were standardised by converting relevant numeric fields from floats (decimals) to integers (whole numbers), and categorical fields, including satisfaction and typical visit time, were changed to ordered or nominal category types as appropriate.

4.2 Justification of Decisions

Each transformation was applied with a clear rationale. Irrelevant columns were dropped to maintain focus. Fuzzy matching was preferred over manual correction for high-cardinality text fields due to the volume of inconsistent entries. Median imputation was chosen for skewed numerical fields to avoid distortion from extreme values, while unknown labels were used for categorical missingness to preserve row counts without introducing misleading assumptions. Clipping rather than removal was applied to outliers where the records were otherwise complete

and informative, minimising data loss while controlling the range for metrics. Collectively, these steps improved consistency, reduced noise, and prepared the dataset for exploratory and feasibility activities.

5. Learning Task: Regression

The chosen learning task is regression, aimed at predicting a customer's satisfaction score based on their demographics and shopping behaviour. Because the target variable, satisfaction, was originally stored as an ordered categorical label (Very Low to Very High), it was encoded numerically using an ordinal mapping on a 1-5 scale, making it a suitable target for regression modelling. To build a meaningful feature set, both numerical and categorical predictors were selected. Numerical signals included age, typical queue wait time, percent self-service checkout uses, distance to store, household income, and average time in store, alongside a created variable, total items, which aggregated six product category columns into a single basket size measure. Typical shopping mission was used as a categorical predictor to capture behavioural context. Scatter plots with regression trend lines were used to visually assess the linear relationship between each numerical predictor and the satisfaction score, providing an early indication of which variables carry predictive signal (**Appendix C**). Regression is appropriate here because the ordinal encoding of satisfaction produces a ranked, numeric target with a natural ordering. However, a key challenge is that satisfaction is subjective and influenced by many unobserved factors, so the available predictors may explain only a modest variance. A complementary text analysis, tagging low-satisfaction customer comments with complaint themes such as Checkout, Staff, and Pricing, reinforces the data suitability discussion by confirming that measurable behavioural variables align with the concerns customers themselves raise. Converting these themes into usable model features represents a further opportunity. An advanced sentiment analysis could be useful to explore shoppers' satisfaction and experience.

6. Findings and Feasibility Assessment

6.1 Key Findings

The dataset remained broadly consistent, with the majority of customers reporting a medium level of satisfaction (**Appendix B, Figure 9**). Several findings aligned with expectations: longer queue wait times, greater distance to the store, and extended time spent in the store were all associated with lower satisfaction scores (**Appendix C**). A more notable finding was that customers with higher household incomes tended to report lower satisfaction, a pattern that warrants further investigation (**Appendix C**). Following data cleaning and transformation, shoppers aged 40-49 and 50-59 represented the largest age group in the dataset (**Appendix B, Figure 3**). Age groups were structured to reflect meaningful life stages: young adults and students (20-24), early adulthood (25-29 and 30-39), middle age (40-49 and 50-59), and shoppers, including possible retirees and seniors (60-74 and 75+). In terms of purchasing behaviour, fruit and vegetables and cupboard staples were the most frequently purchased categories, while meat, frozen foods, toiletries, and miscellaneous items were acquired less often (**Appendix B, Figure 7-8**). Weekly and top-up shoppers were more prevalent than bulk buyers, reflecting a pattern of regular, moderate volume trips likely consistent with smaller household sizes and fewer dependants (**Appendix B, Figure 11**). An analysis of income by store location revealed notable geographic variation. Customers at Dereham, Wymondham, and Thetford reported slightly higher average household incomes, while Fakenham and Norwich shoppers reported comparatively lower incomes (**Appendix B, Figure 13**). Correspondingly, the highest

concentrations of very low to low satisfaction were recorded at Thetford and Dereham, while Fakenham and Norwich showed a greater proportion of high to very high satisfaction responses, consistent with the broader finding that higher income customers tend to report lower satisfaction (**Appendix B, Figure 10**).

6.2 Feasibility Assessment

The dataset provides sufficient structure and behavioural variation to support the proposed regression task. The availability of demographic, transactional, and experiential variables across a meaningful sample size strengthens its suitability for early-stage modelling. A challenge encountered during analysis was designing visualisations that are both accessible to a broad audience. Careful consideration of axis scaling was required to ensure that the ranges chosen accurately reflect the underlying data, rather than inadvertently emphasising or downplaying patterns in a way that could mislead interpretation.

6.3 Recommended Additional Variables

Several additional variables would enrich future analysis. These include whether a customer received assistance from a store employee and how quickly that help was provided; shopping frequency expressed as visits per week; and a customer's typical mode of transportation, including the distance between their nearest public transport stop and the store. Each of these would offer meaningful context for explaining variation in satisfaction and shopping behaviour.

7. Conclusion

The analysis presented several patterns across customer demographics, purchasing behaviour, and satisfaction levels. The middle age group represented the slightly larger customer segment, and basket compositions were consistent with regular top-up or weekly shopping rather than bulk buying, as expected for a conventional supermarket. Household composition skewed toward fewer dependants, which may partly explain this trend. Fruit and vegetables ranked among the most frequently purchased categories, reflecting positive purchasing habits within the customer base. A notable theme was the inverse relationship between household income and satisfaction, with higher-income customers consistently reporting lower scores. This pattern was further reflected in location analysis, where stores serving higher-income areas tended to record lower satisfaction. Individual stores would benefit from closely reviewing customer comments to identify areas for improvement. Regarding data suitability, the dataset proved sufficiently useful to support exploratory analysis and the proposed regression task. However, there are limitations; key variables such as employee interaction quality, shopping frequency, and transportation access were absent. The inclusion of these fields in future data collection would strengthen the analysis and predictive values of the dataset.

Setup and Imports

This section imports all required libraries for data analysis, loads the dataset into a DataFrame, and defines a utility function that logs completed tasks to a separate .txt file for documentation and reproducibility.

In [337...

```
#Import libraries
import pandas as pd
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
```

In [338...

```
# Function to record tasks into Travel_Log.txt
from datetime import datetime, timezone

def tlog(task_completed, why, tools_used="", notes="", file="travel_log.txt"):
    ts = datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M UTC")

    entry = (
        f" [{ts}]\n"
        f"Task completed: {task_completed}\n"
        f"Why I did it: {why}\n"
        f"Tools/commands used (optional): {tools_used}\n"
        f"Notes: {notes}\n\n")
    with open(file, "a", encoding="utf-8") as f:
        f.write(entry)

#Example of cell log to record each task:
#tlog(
#    #task_completed="text",
#    #why="text",
#    #tools_used="text",
#    #notes="text")
```

In [339...

```
# Import and read Supermarket data
supermarket = pd.read_csv("SupermarketEvaluation.csv")
```

Preliminary Data Examination

This includes data familiarization and descriptive stats to understand each field: Range, mean, median, SD.

- Records and # fields (attributes)
- Proportion of fields with missing values

In [340...

```
supermarket.sample(10) #Random sample to inspect data
```

Out[340...

	age	household_income	gender	store_location	typical_queue_wait_time	distance_to_store	percent_self_service_checkout_uses	children	fruit_and_veg_items	meat_items	...	other_items	has_loyalty_card	brand_loyalty	typical_visit_time	typical_shopping_mission	brings_own_bag
4218	61	44999.0	Male	Wymondham	17.3	2.4	0.00	0	8	5	...	3	No	High	Morning	Weekly shop	Yes
1059	53	73694.0	Female	Norwich	14.9	8.1	0.02	1	10	1	...	4	No	Medium	Evening	Top-up	Yes
3525	31	NaN	Non-binary	Norwich	16.5	NaN	0.17	2	8	7	...	1	No	Medium	Afternoon	Weekly shop	Yes
2961	52	107765.0	Female	Thetford	8.7	3.1	0.14	1	8	2	...	4	No	Medium	Morning	Top-up	Yes
4234	51	53405.0	Female	Norwich	6.3	0.7	0.52	0	9	3	...	7	Yes	Low	Afternoon	Weekly shop	No
4451	70	62451.0	Non-binary	Wymondham	17.0	5.6	0.00	1	13	1	...	4	No	High	Evening	Top-up	No
4429	42	45771.0	Male	Thetford	18.4	1.4	0.09	2	10	2	...	7	No	Medium	Morning	Top-up	Yes
4665	20	NaN	Non-binary	Wymondham	8.3	3.8	0.61	0	6	2	...	1	Yes	Medium	Evening	Weekly shop	Yes
2433	54	32908.0	Male	Wymondham	14.5	NaN	0.05	1	9	2	...	5	No	Medium	Evening	Weekly shop	Yes
2022	34	50949.0	Male	Fakenham	17.9	1.3	0.19	0	7	1	...	3	No	Low	Afternoon	Top-up	Yes

10 rows × 23 columns

In [341...

```
supermarket.info() #Basic information including data types and NaNs
```



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 23 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   age                                    5000 non-null   int64
1   household_income                      4812 non-null   float64
2   gender                                4919 non-null   object
3   store_location                        5000 non-null   object
4   typical_queue_wait_time              4952 non-null   float64
5   distance_to_store                    4088 non-null   float64
6   percent_self_service_checkout_uses   5000 non-null   float64
7   children                              5000 non-null   int64
8   fruit_and_veg_items                  5000 non-null   int64
9   meat_items                           5000 non-null   int64
10  cupboard_items                       5000 non-null   int64
11  frozen_items                         4905 non-null   float64
12  toiletries_items                     4941 non-null   float64
13  other_items                          5000 non-null   int64
14  has_loyalty_card                     5000 non-null   object
15  brand_loyalty                        5000 non-null   object
16  typical_visit_time                   5000 non-null   object
17  typical_shopping_mission             5000 non-null   object
18  brings_own_bag                       5000 non-null   object
19  average_time_in_store                5000 non-null   float64
20  typical_payment_method               4801 non-null   object
21  satisfaction                          5000 non-null   object
22  Customer_Comment                    3423 non-null   object
dtypes: float64(7), int64(6), object(10)
memory usage: 898.6+ KB
```

```
In [342... supermarket.describe() #Summary stats
```

Out[342...

	age	household_income	typical_queue_wait_time	distance_to_store	percent_self_service_checkout_uses	children	fruit_and_veg_items	meat_items	cupboard_items	frozen_items	toiletries_items	other_items	average_time_in_store
count	5000.000000	4.812000e+03	4952.000000	4088.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	4905.000000	4941.000000	5000.000000	5000.000000
mean	48.790800	6.474890e+04	41.797920	1.954232	0.331838	0.756400	7.749800	4.123200	6.774400	3.974924	2.364906	3.079800	66.233060
std	33.874901	3.969892e+05	466.637391	1.330235	0.250311	0.924137	4.306984	3.682001	6.124198	3.732761	2.780538	2.034075	18.235361
min	20.000000	5.081000e+03	2.200000	0.300000	0.000000	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	27.000000
25%	31.000000	2.546500e+04	9.600000	1.100000	0.100000	0.000000	5.000000	1.000000	2.000000	1.000000	0.000000	2.000000	53.000000
50%	46.000000	3.595550e+04	12.000000	1.600000	0.300000	0.000000	7.000000	3.000000	5.000000	3.000000	1.000000	3.000000	63.700000
75%	60.000000	5.202925e+04	14.500000	2.300000	0.530000	1.000000	10.000000	6.000000	9.000000	6.000000	3.000000	4.000000	76.600000
max	367.000000	1.003282e+07	9920.400000	16.500000	0.970000	3.000000	39.000000	27.000000	54.000000	27.000000	20.000000	20.000000	184.300000

```
In [343... supermarket.shape #Check for number of rows/columns
```

Out[343... (5000, 23)

```
In [344... supermarket[supermarket.duplicated()] #Check for duplicate rows
```

Out[344...

	age	household_income	gender	store_location	typical_queue_wait_time	distance_to_store	percent_self_service_checkout_uses	children	fruit_and_veg_items	meat_items	...	other_items	has_loyalty_card	brand_loyalty	typical_visit_time	typical_shopping_mission	brings_own_bag	average_time_in_store
0 rows × 23 columns																		

```
In [345... print(supermarket.isnull().sum()) #Check for missing values
print(supermarket.isnull().sum() * 100 / len(supermarket)) #Percentage
```

```
age 0
household_income 188
gender 81
store_location 0
typical_queue_wait_time 48
distance_to_store 912
percent_self_service_checkout_uses 0
children 0
fruit_and_veg_items 0
meat_items 0
cupboard_items 0
frozen_items 95
toiletries_items 59
other_items 0
has_loyalty_card 0
brand_loyalty 0
typical_visit_time 0
typical_shopping_mission 0
brings_own_bag 0
average_time_in_store 0
typical_payment_method 199
satisfaction 0
Customer_Comment 1577
dtype: int64
age 0.00
household_income 3.76
gender 1.62
store_location 0.00
typical_queue_wait_time 0.96
distance_to_store 18.24
percent_self_service_checkout_uses 0.00
children 0.00
fruit_and_veg_items 0.00
meat_items 0.00
cupboard_items 0.00
frozen_items 1.90
toiletries_items 1.18
other_items 0.00
has_loyalty_card 0.00
brand_loyalty 0.00
typical_visit_time 0.00
typical_shopping_mission 0.00
brings_own_bag 0.00
average_time_in_store 0.00
typical_payment_method 3.98
satisfaction 0.00
Customer_Comment 31.54
dtype: float64

In [346... supermarket.nunique() #Check unique values in each column

Out[346... age 109
household_income 4661
gender 5
store_location 162
typical_queue_wait_time 258
distance_to_store 91
percent_self_service_checkout_uses 98
children 4
fruit_and_veg_items 31
meat_items 27
cupboard_items 41
frozen_items 27
toiletries_items 21
other_items 18
has_loyalty_card 2
brand_loyalty 3
typical_visit_time 3
typical_shopping_mission 40
brings_own_bag 2
average_time_in_store 835
typical_payment_method 17
satisfaction 5
Customer_Comment 3423
dtype: int64

In [347... print(supermarket.shape) #Check value counts
supermarket['gender'].value_counts()

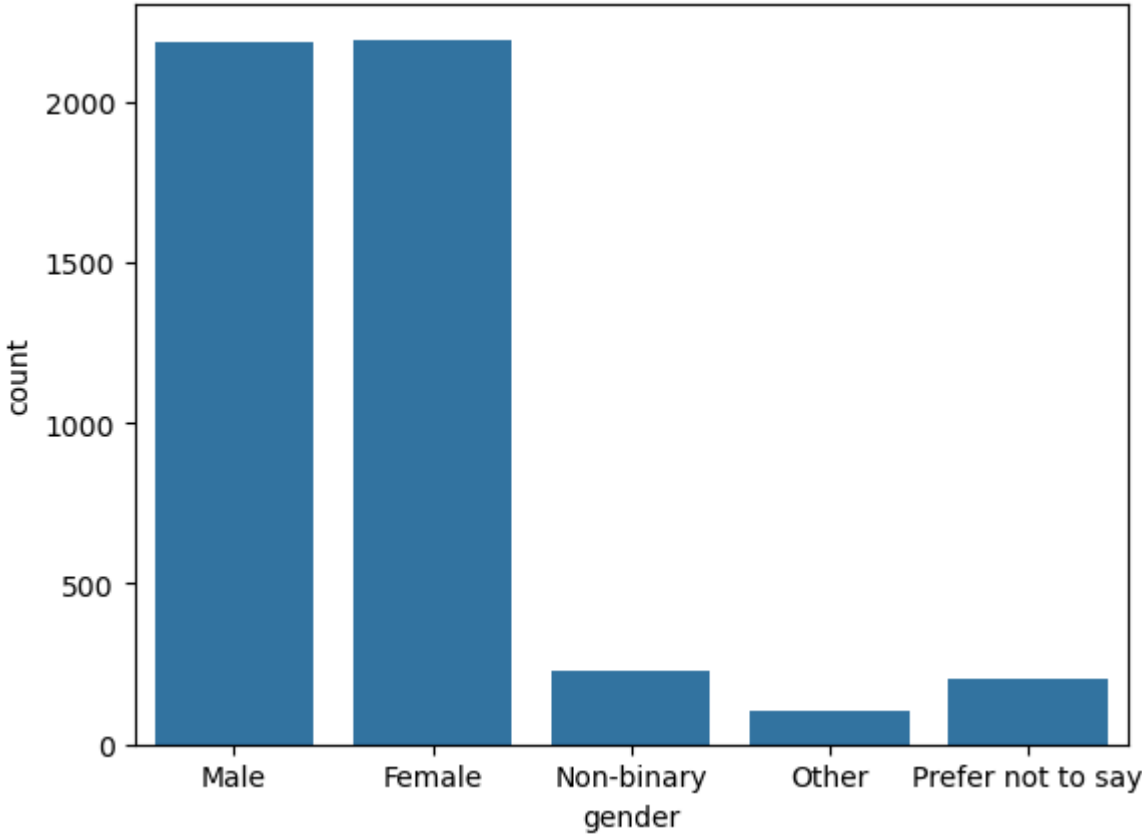
(5000, 23)

Out[347... gender
Female 2192
Male 2185
Non-binary 231
Prefer not to say 206
Other 105
Name: count, dtype: int64
```



```
In [348... sns.countplot(x='gender' , data = supermarket) #Countplot of gender (Figure 1)
```

```
Out[348... <Axes: xlabel='gender', ylabel='count'>
```



```
In [349... supermarket['age'].describe() #Retrieving age stats
```

```
Out[349... count    5000.000000
mean      48.790800
std       33.874901
min       20.000000
25%       31.000000
50%       46.000000
75%       60.000000
max       367.000000
Name: age, dtype: float64
```

```
In [350... supermarket['age'].median() #Median age
```

```
Out[350... 46.0
```

```
In [351... supermarket['household_income'].describe()
```

```
Out[351... count    4.812000e+03
mean     6.474890e+04
std      3.969892e+05
min      5.081000e+03
25%      2.546500e+04
50%      3.595550e+04
75%      5.202925e+04
max      1.003282e+07
Name: household_income, dtype: float64
```

```
In [352... #Check for unreasonable ages
age_outliers = supermarket[(supermarket['age'] < 18) | (supermarket['age'] > 120)]
age_outliers.head()['age']
```

```
Out[352... 34    283
39    325
41    332
92    335
161   225
Name: age, dtype: int64
```

```
In [353... #Find income outliers using IQR method (Copilot generated)
Q1 = supermarket['household_income'].quantile(0.25)
Q3 = supermarket['household_income'].quantile(0.75)
IQR = Q3 - Q1
income_outliers = supermarket[(supermarket['household_income'] < (Q1 - 1.5 * IQR)) |
                               (supermarket['household_income'] > (Q3 + 1.5 * IQR))]

income_outliers.head()['household_income']
```

```
Out[353... 9      94444.0
46     150327.0
53    10032823.0
64     129262.0
74     131325.0
Name: household_income, dtype: float64
```

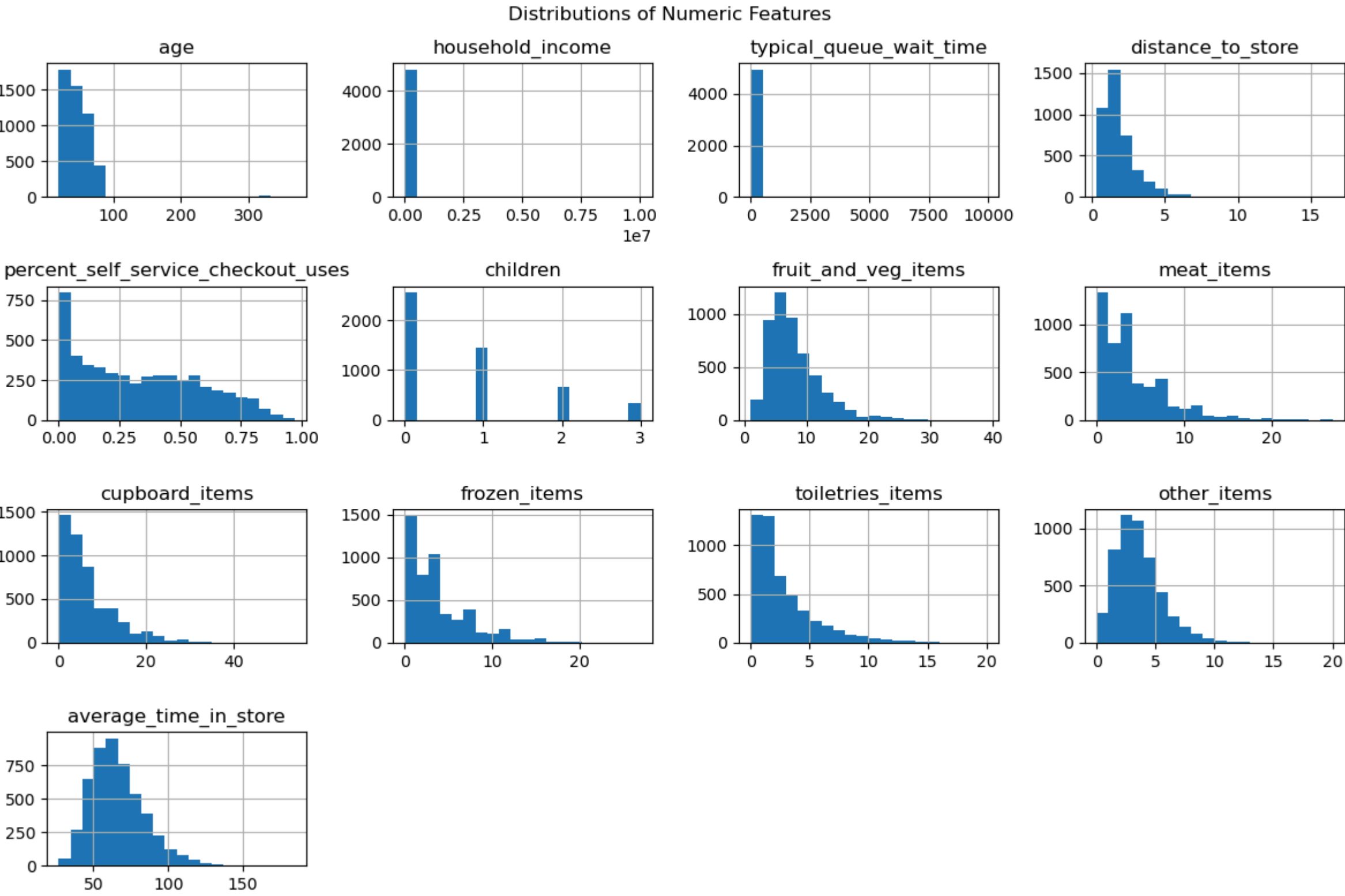
```
In [354... # Extracting categorical and numerical features (Retrieved from Youtube video)
# https://youtu.be/lXcBYSN67ig?si=dKNIHP_y_cEfEeS7
```

```
categorical_features = supermarket.select_dtypes(include=['object', 'category']).columns.tolist()
numerical_features = supermarket.select_dtypes(include=['int64', 'float64']).columns.tolist()

print("Categorical Features:", categorical_features)
print("Numerical Features:", numerical_features)
```

Categorical Features: ['gender', 'store_location', 'has_loyalty_card', 'brand_loyalty', 'typical_visit_time', 'typical_shopping_mission', 'brings_own_bag', 'typical_payment_method', 'satisfaction', 'Customer_Comment']
Numerical Features: ['age', 'household_income', 'typical_queue_wait_time', 'distance_to_store', 'percent_self_service_checkout_uses', 'children', 'fruit_and_veg_items', 'meat_items', 'cupboard_items', 'frozen_items', 'toiletries_items', 'other_items', 'average_time_in_store']

```
In [355... # Histograms of numerical features
#(Retrieved from Youtube video) (Figure 2)
# https://youtu.be/LXcBYSN67ig?si=dKNIHP_y_cEfEeS7
supermarket[numerical_features].hist(bins=20, figsize=(12, 8))
plt.suptitle("Distributions of Numeric Features")
plt.tight_layout()
plt.show()
```



Data Cleansing

Preparing and transforming data for later learning stages by improving quality.

Key Activities:

- String cleaning
- Manage NaNs and missing data
- Handle outliers
- Balance target variables and data types

String Cleaning

Ensure that all manually entered text values are accurately spelled to support reliable data analysis.

```
In [356... #Removing columns that are not useful (New dataframe variable)
df = supermarket.drop(columns=["brings_own_bag", "brand_loyalty"])
```

```
In [357... #Use of fuzzy matching to ensure correct city spelling (Copilot generated)
##Imports a library that finds closest matching strings
from difflib import get_close_matches
#Correct city spelling into variable
valid_cities = ["Wymondham", "Fakenham", "Dereham", "Norwich", "Thetford"]

#Function to look for the closest match (must be at least 75% similar)
def fix_city(x):
    match = get_close_matches(str(x), valid_cities, n=1, cutoff=0.75)
    return match[0] if match else x

df['store_location'] = df['store_location'].apply(fix_city)
df['store_location'].value_counts() #Check for misspellings
```

Out[357... store_location
Wymondham 1058
Fakenham 1031
Norwich 1025
Dereham 1010
Thetford 876
Name: count, dtype: int64

```
In [358... df['typical_payment_method'].unique() #Shows all unique values for typical payment method
```

Out[358... array(['Cash', nan, 'Google Pay', 'Card', 'Apple Pay', 'Unknown', 'Cad',
'Cajrd', 'Cas', 'ard', 'Crd', 'Cakrd', 'Crard', 'Casth', 'Cardf',
'Caard', 'Google Payn', 'wCard'], dtype=object)

```
In [359... # Fix common typos (Manual fix since there are not a lot of incorrect values)
df['typical_payment_method'] = (
    df['typical_payment_method']
    .astype("string") # make everything a string
    .str.strip()      # remove spaces before/after text
    .replace({
        "Cad": "Card",
        "Cajrd": "Card",
        "Cas": "Cash",
        "Casth": "Cash",
        "ard": "Card",
        "Crd": "Card",
        "Cakrd": "Card",
        "Crard": "Card",
        "Cardf": "Card",
        "Caard": "Card",
        "wCard": "Card",
        "Google Payn": "Google Pay"})

df['typical_payment_method'].value_counts()
```

Out[359... typical_payment_method
Card 2738
Cash 1018
Apple Pay 586
Google Pay 263
Unknown 196
Name: count, dtype: Int64

```
In [360... #Use of fuzzy matching to ensure correct mission spelling
#Correct city spelling into variable
valid_missions = ["Weekly shop", "Top-up", "Bulk shop"]

#Function to look for the closest match (must be at least 75% similar)
def fix_mission(x):
    match = get_close_matches(str(x), valid_missions, n=1, cutoff=0.75)
    return match[0] if match else x

df['typical_shopping_mission'] = df['typical_shopping_mission'].apply(fix_mission)
df['typical_shopping_mission'].value_counts() #Check for misspellings
```

Out[360... typical_shopping_mission
Top-up 2085
Weekly shop 2068
Bulk shop 847
Name: count, dtype: int64

NaNs/Null Values

Imputing missing values.

```
In [361... print(df['Customer_Comment'].isna().sum()) #Check for count of NaNs
print(df['Customer_Comment'].notna().sum()) #Check for count of inputs

1577
3423
```



```
In [362... #Fill in NaNs for Customer Comment (useful when preparing text data)
df['Customer_Comment'] = df['Customer_Comment'].fillna('')
df['Customer_Comment'].isna().sum()

Out[362... np.int64(0)

In [363... #Fill in blanks with Unknown
df['typical_payment_method'] = df['typical_payment_method'].fillna('Unknown')
df['gender'] = df['gender'].fillna('Unknown')

In [364... #List of numeric columns to replace NaNs in each column with median
num_cols = ['household_income', 'typical_queue_wait_time',
            'distance_to_store', 'frozen_items', 'toiletries_items']
df[num_cols] = df[num_cols].fillna(df[num_cols].median())

df.isnull().sum()

Out[364... age                0
household_income         0
gender                   0
store_location           0
typical_queue_wait_time  0
distance_to_store        0
percent_self_service_checkout_uses  0
children                 0
fruit_and_veg_items      0
meat_items               0
cupboard_items           0
frozen_items             0
toiletries_items         0
other_items              0
has_loyalty_card         0
typical_visit_time       0
typical_shopping_mission 0
average_time_in_store    0
typical_payment_method   0
satisfaction             0
Customer_Comment        0
dtype: int64

Removing Outliers

Remove outliers to improve quality of data analysis.

In [365... lower, upper = 18, 120 #Setting age parameter

df['age'] = supermarket['age'].clip(lower=lower, upper=upper)
df['age'].describe()

Out[365... count      5000.000000
mean       46.431000
std        19.354869
min        20.000000
25%        31.000000
50%        46.000000
75%        60.000000
max        120.000000
Name: age, dtype: float64

In [366... lower = Q1 - 1.5 * IQR #Setting income parameter
upper = Q3 + 1.5 * IQR

df['household_income'] = df['household_income'].clip(lower=lower, upper=upper)
df['household_income'].describe()

Out[366... count      5000.000000
mean     40804.798050
std     20741.169082
min      5081.000000
25%     25870.000000
50%     35955.500000
75%     51354.000000
max      91875.625000
Name: household_income, dtype: float64

In [367... #Checking for unreasonable queue wait time values
#Checks sum of values that are greater than 1200 seconds (20 min.)
#Checks sum of values that are greater than 1800 seconds (30 min.)
print((df['typical_queue_wait_time'] > 1200).sum())
print((df['typical_queue_wait_time'] > 1800).sum())

23
22
```

```
In [368... #Setting queue wait time parameter
df = df[df['typical_queue_wait_time'] <= 1800]
df['typical_queue_wait_time'].describe()
```

```
Out[368... count      4978.000000
mean       12.557995
std        23.861344
min         2.200000
25%         9.600000
50%        12.000000
75%        14.400000
max        1674.100000
Name: typical_queue_wait_time, dtype: float64
```

```
In [369... df['average_time_in_store'].describe() #Check time in store
```

```
Out[369... count      4978.000000
mean       66.254158
std        18.240746
min        27.000000
25%        53.025000
50%        63.700000
75%        76.600000
max        184.300000
Name: average_time_in_store, dtype: float64
```

```
In [370... #Check for unreasonable outliers of numercial features
df[['age', 'household_income', 'typical_queue_wait_time', 'distance_to_store',
    'percent_self_service_checkout_uses', 'children', 'fruit_and_veg_items',
    'meat_items', 'cupboard_items', 'frozen_items',
    'toiletries_items', 'other_items', 'average_time_in_store']].describe()
```

Out[370...		age	household_income	typical_queue_wait_time	distance_to_store	percent_self_service_checkout_uses	children	fruit_and_veg_items	meat_items	cupboard_items	frozen_items	toiletries_items	other_items	average_time_in_store
	count	4978.000000	4978.000000	4978.000000	4978.000000	4978.000000	4978.000000	4978.000000	4978.000000	4978.000000	4978.000000	4978.000000	4978.000000	4978.000000
	mean	46.429490	40830.237369	12.557995	1.890900	0.331754	0.755725	7.747690	4.122338	6.779229	3.959823	2.349538	3.078144	66.254158
	std	19.374443	20744.700200	23.861344	1.211909	0.250485	0.923534	4.304399	3.677050	6.125969	3.701651	2.769339	2.034690	18.240746
	min	20.000000	5081.000000	2.200000	0.300000	0.000000	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	27.000000
	25%	31.000000	25891.750000	9.600000	1.200000	0.100000	0.000000	5.000000	1.000000	2.000000	1.000000	0.000000	2.000000	53.025000
	50%	46.000000	35955.500000	12.000000	1.600000	0.300000	0.000000	7.000000	3.000000	5.000000	3.000000	1.000000	3.000000	63.700000
	75%	60.000000	51372.500000	14.400000	2.100000	0.530000	1.000000	10.000000	6.000000	9.000000	5.000000	3.000000	4.000000	76.600000
	max	120.000000	91875.625000	1674.100000	16.500000	0.970000	3.000000	39.000000	27.000000	54.000000	27.000000	20.000000	20.000000	184.300000

Data Types

In this section, column data types can be changed to improve performance, clarity, and consistency.

```
In [371... #Changed data type to integer
df = df.astype({'frozen_items':'int','toiletries_items':'int'})
```

```
In [372... #Changed data type to category and bool
df = df.astype({
    'typical_payment_method':'category',
    'gender':'category','store_location':'category',
    'typical_shopping_mission':'category'})
```

```
In [373... #Changed data type to ordered categories
df['satisfaction'] = df['satisfaction'].astype(
    pd.CategoricalDtype(['Very Low', 'Low', 'Medium', 'High', 'Very High'], ordered=True))

df['typical_visit_time'] = df['typical_visit_time'].astype(
    pd.CategoricalDtype(['Morning','Afternoon','Evening'], ordered=True))

df.dtypes
```

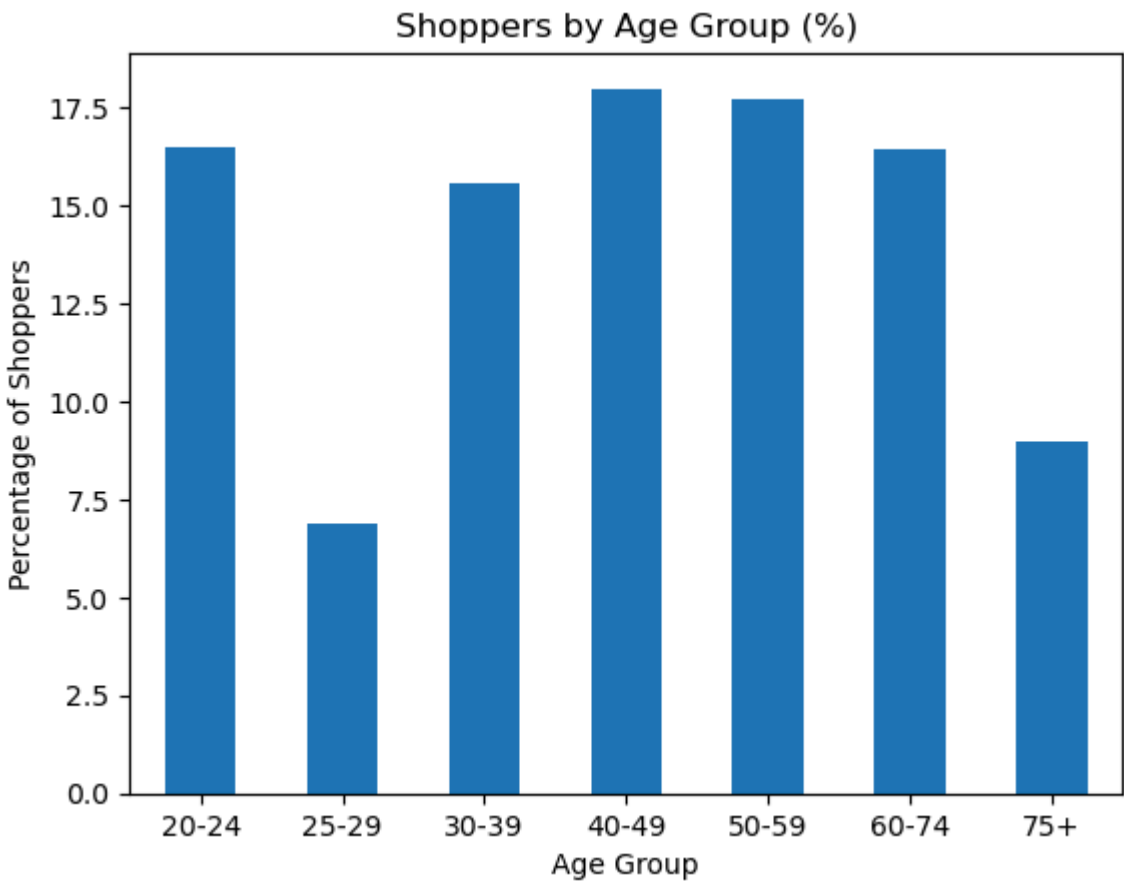
```
Out[373... age                                int64
household_income          float64
gender                    category
store_location            category
typical_queue_wait_time   float64
distance_to_store         float64
percent_self_service_checkout_uses float64
children                  int64
fruit_and_veg_items       int64
meat_items                int64
cupboard_items            int64
frozen_items              int64
toiletries_items          int64
other_items               int64
has_loyalty_card          object
typical_visit_time        category
typical_shopping_mission  category
average_time_in_store     float64
typical_payment_method    category
satisfaction              category
Customer_Comment          object
dtype: object
```

Exploratory Visualisations

Post-transformation visualisations that clearly explain the data and its suitability for a learning task.

```
In [374... # Bar chart of percentage of how many shoppers in each group in age_pct variable (Figure 3)
age_pct = (
    pd.cut(df['age'], bins=bins, labels=labels, ordered=True, right=False)
    .value_counts(normalize=True)
    .sort_index()
    * 100)

plt.figure()
age_pct.plot(kind='bar')
plt.title('Shoppers by Age Group (%)')
plt.xlabel('Age Group')
plt.ylabel('Percentage of Shoppers')
plt.xticks(rotation=0)
plt.show()
print("bins:", bins)
print("labels:", labels)
age_pct
```

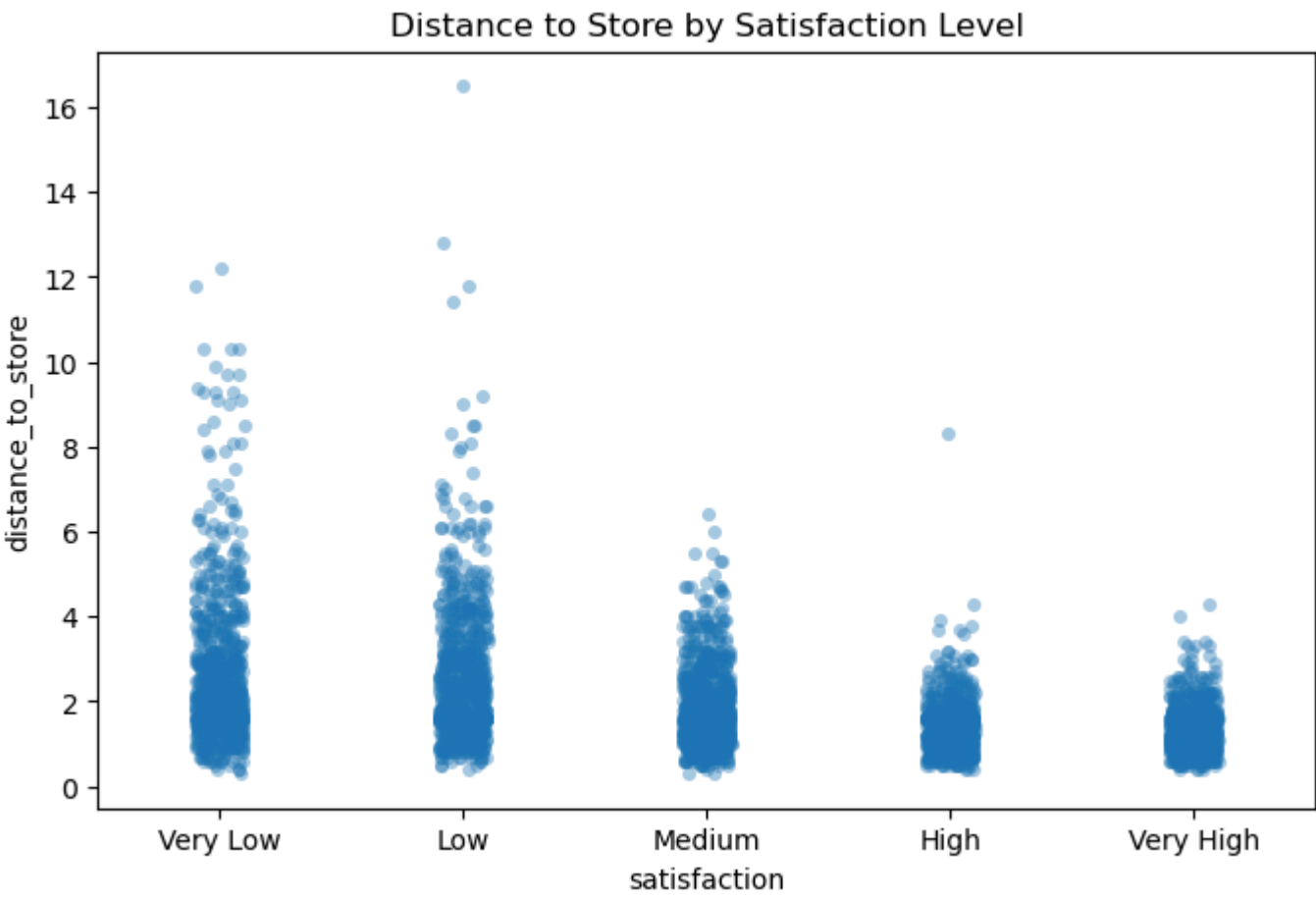


bins: [20, 25, 30, 40, 50, 60, 75, inf]
labels: ['20-24', '25-29', '30-39', '40-49', '50-59', '60-74', '75+']

```
Out[374... age
20-24    16.472479
25-29     6.890317
30-39    15.548413
40-49    17.979108
50-59    17.717959
60-74    16.412214
75+      8.979510
Name: proportion, dtype: float64
```


In [375...

```
#Strip plot of distance to store vs Satisfaction (Figure 4)
plt.figure(figsize=(8,5))
sns.stripplot(data=df, x="satisfaction", y="distance_to_store", alpha=0.4)
plt.title("Distance to Store by Satisfaction Level")
plt.show()
```



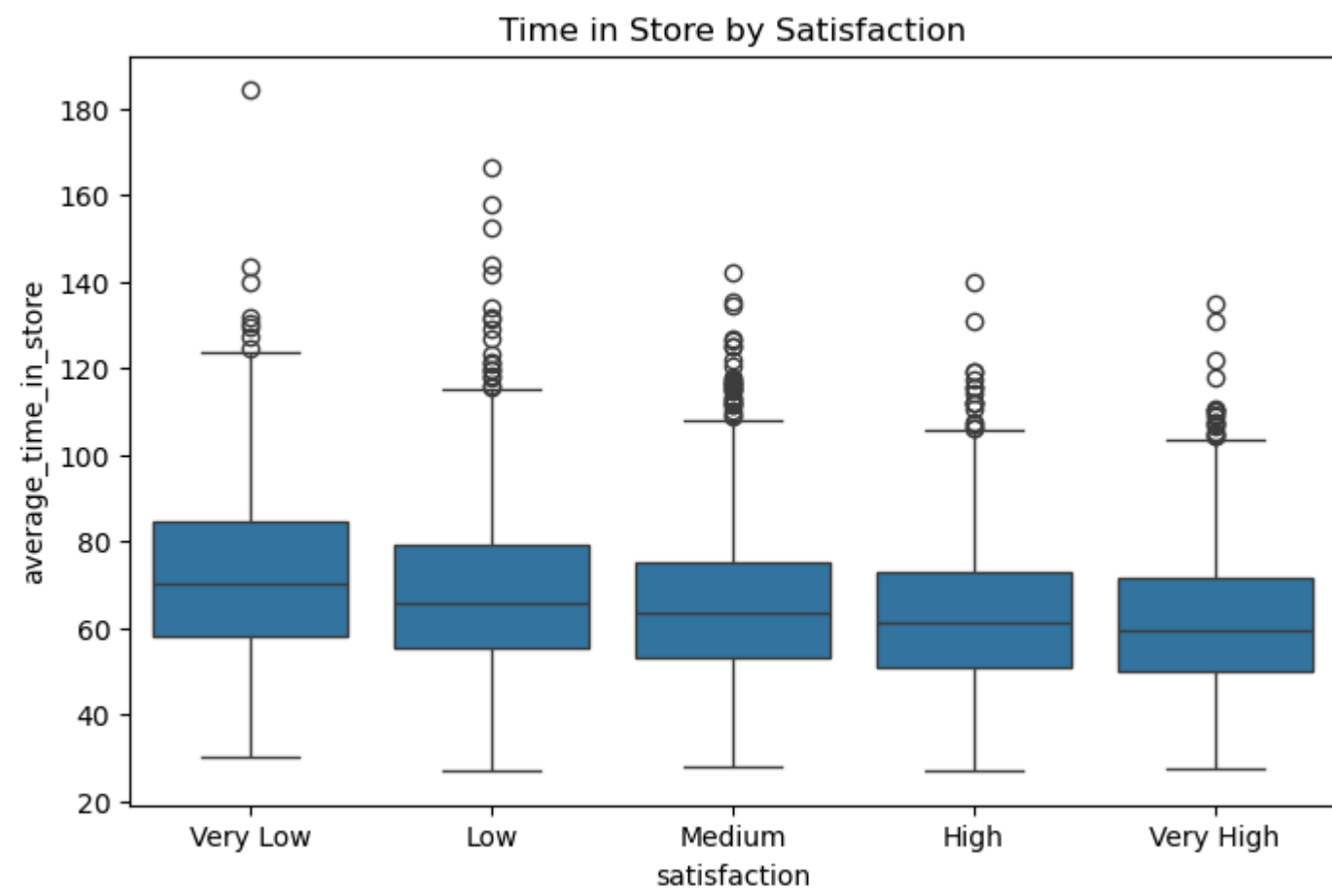
In [376...

```
#Violin and Strip graph comparing time in store and satisfaction (Figure 5)
plt.figure(figsize=(8,5))
sns.violinplot(
    data=df, x="satisfaction", y="average_time_in_store",
    inner=None, cut=0, color="#a6cee3"
)
sns.stripplot(
    data=df, x="satisfaction", y="average_time_in_store",
    color="#1f78b4", alpha=0.35, jitter=0.25, size=3 #Copilot generated
)
plt.title("Time in Store by Satisfaction (Violin + Strip)")
plt.xlabel("Satisfaction")
plt.ylabel("Time in store")
plt.tight_layout()
plt.show()
```



In [377...

```
#Boxplot comparing time in store and satisfaction (Figure 6)
plt.figure(figsize=(8,5))
sns.boxplot(data=df, x="satisfaction", y="average_time_in_store")
plt.title("Time in Store by Satisfaction")
plt.show()
```

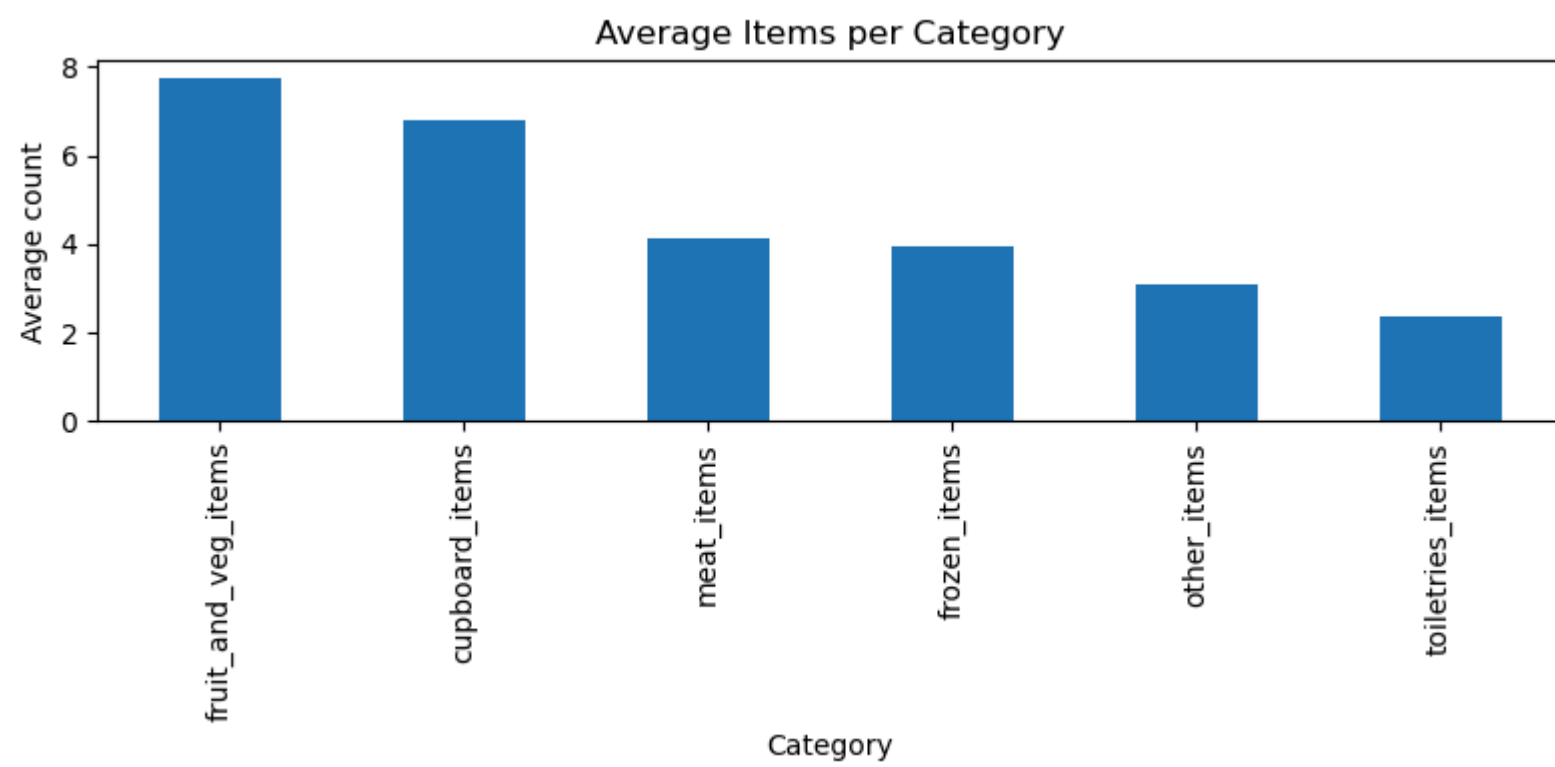


In [378... *#Bar chart of the average number of items across shopping categories (Figure 7)*

```
item_cols = [
    "fruit_and_veg_items",
    "meat_items",
    "cupboard_items",
    "frozen_items",
    "toiletries_items",
    "other_items"]

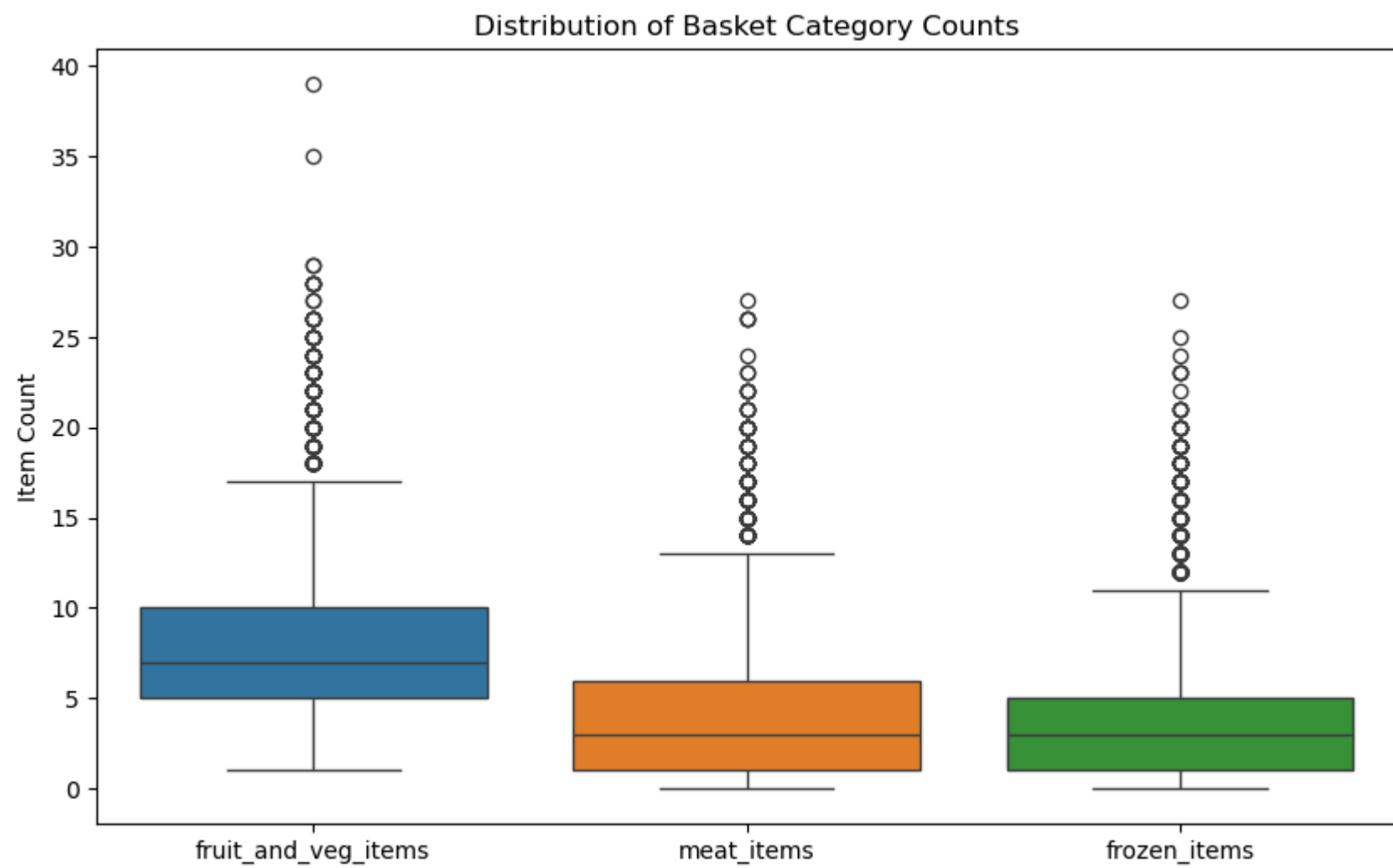
avg_items = df[item_cols].mean().sort_values(ascending=False)

plt.figure(figsize=(8,4))
avg_items.plot(kind="bar")
plt.title("Average Items per Category")
plt.xlabel("Category")
plt.ylabel("Average count")
plt.tight_layout()
plt.show()
```



In [379... *#Boxplot of basket composition (fruit/veg, meat, frozen food)(Figure 8)*

```
plt.figure(figsize=(10,6))
sns.boxplot(data=df[["fruit_and_veg_items", "meat_items", "frozen_items"]])
plt.title("Distribution of Basket Category Counts")
plt.ylabel("Item Count")
plt.show()
```

```
In [380... #Count plot of Satisfaction (Figure 9)
plt.figure(figsize=(6,3))
sns.countplot(data=df, x="satisfaction", order=df["satisfaction"].value_counts().index)
plt.title("Satisfaction Levels")
plt.xlabel("Satisfaction")
plt.ylabel("Count")
plt.show()
```



```
In [381... #Count plot of satisfaction by store location (Figure 10)
plt.figure(figsize=(12, 6))

#Plot with accessible spacing and colors
ax = sns.countplot(
    data=df,
    x="store_location",
    hue="satisfaction",
    palette="colorblind")

plt.title("Satisfaction Levels by Store Location", fontsize=14)
plt.xlabel("Store Location", fontsize=12)
plt.ylabel("Number of Customers", fontsize=12)

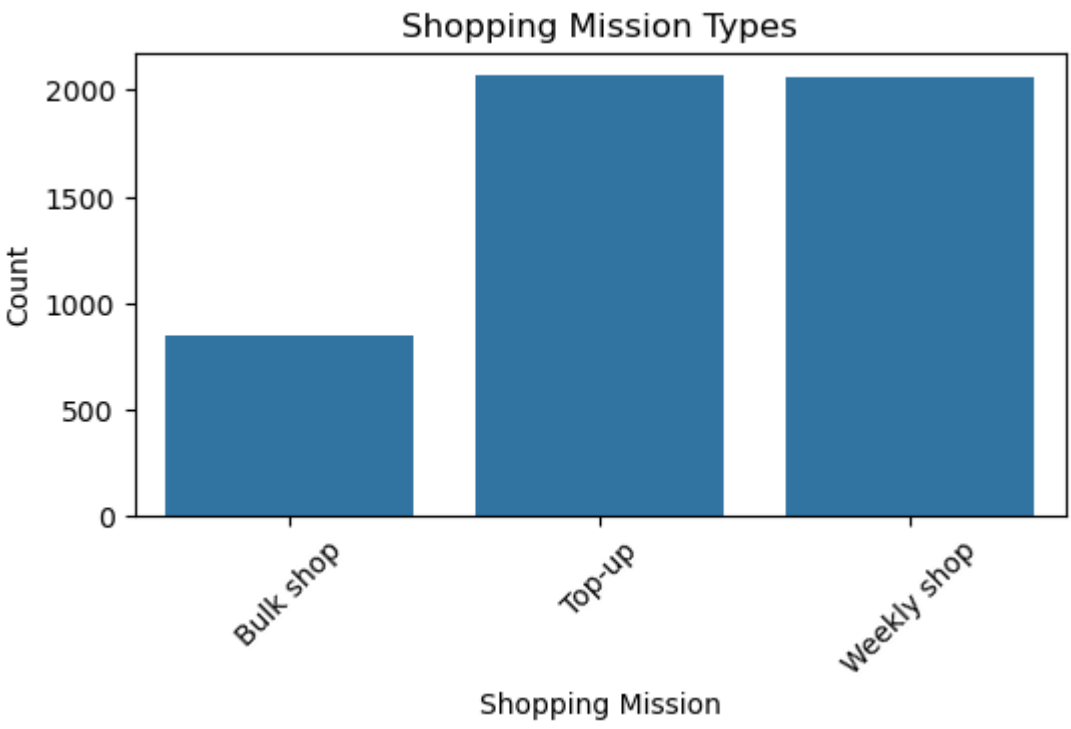
#Legend
plt.legend(title="Satisfaction Level", title_fontsize=12)

#Add value labels on top of each bar
for container in ax.containers:
    ax.bar_label(container, fmt='%d')

plt.tight_layout()
plt.show()
```



```
In [382... #Bar chart of shopping mission (Figure 11)
plt.figure(figsize=(6,3))
sns.countplot(data=df, x="typical_shopping_mission")
plt.xticks(rotation=45)
plt.title("Shopping Mission Types")
plt.xlabel("Shopping Mission")
plt.ylabel("Count")
plt.show()
```

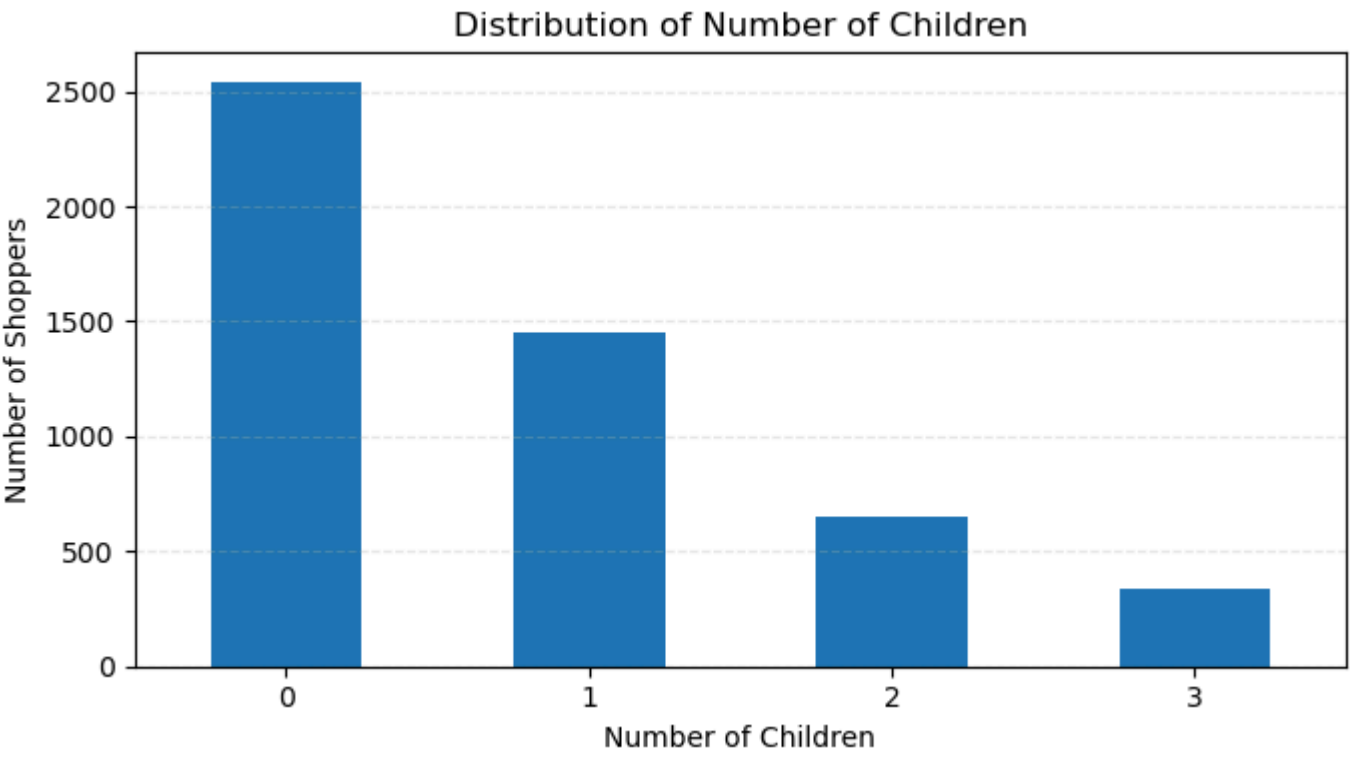


```
In [383... #Bar chart of number of children for every shopper (Figure 12)
child_counts = (df['children']
                .value_counts()
                .sort_index())

plt.figure(figsize=(7,4))
ax = child_counts.plot(kind='bar')

ax.set_title('Distribution of Number of Children')
ax.set_xlabel('Number of Children')
ax.set_ylabel('Number of Shoppers')
plt.xticks(rotation=0)

plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```



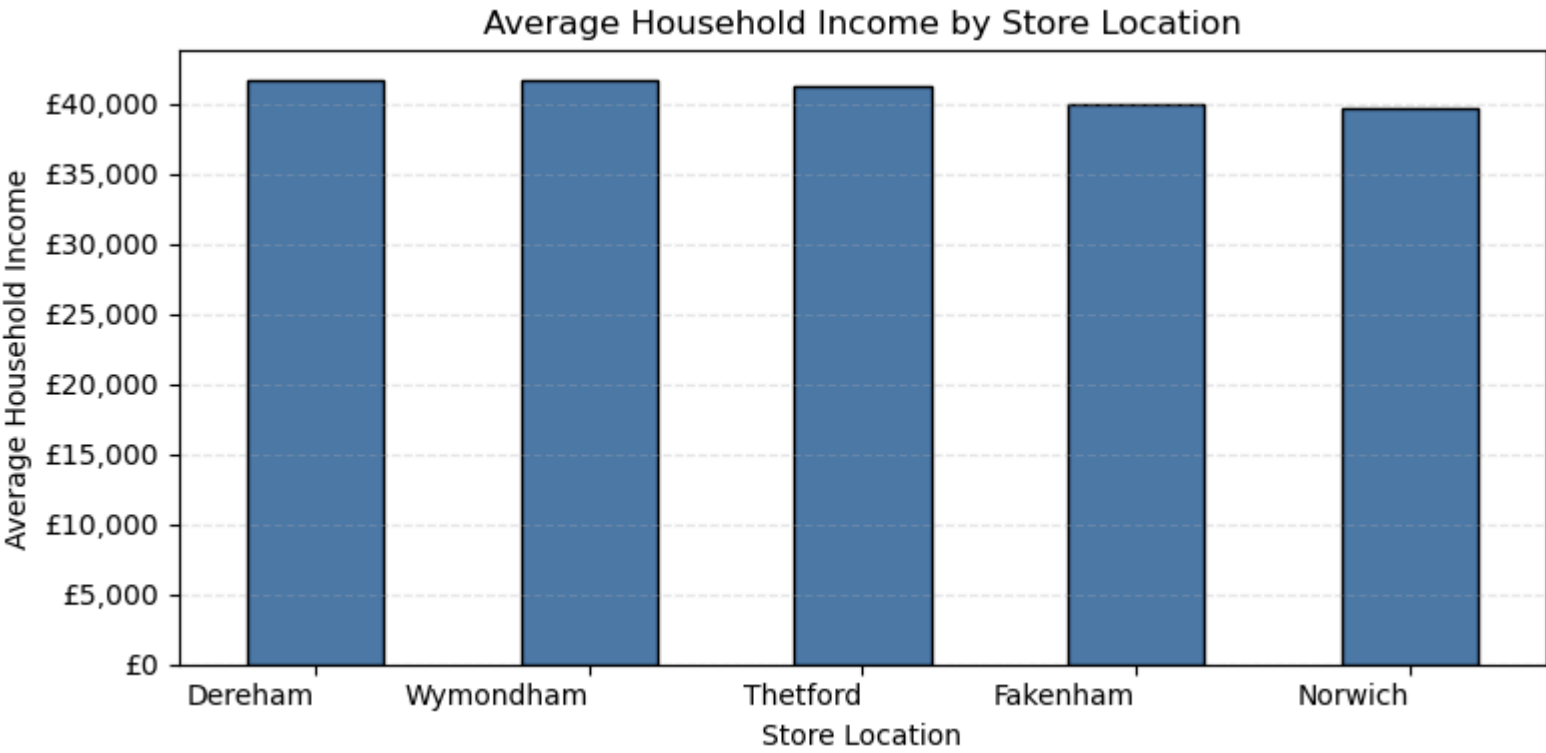
```
In [384... import matplotlib.ticker as mtick
#Bar chart of household income mean income by store location (Figure 13)
income_by_store = (
    df.groupby('store_location', observed=True)['household_income']
      .mean()
      .sort_values(ascending=False))

plt.figure(figsize=(8, 4))
ax = income_by_store.plot(kind='bar', color='#4C78A8', edgecolor='black')

ax.set_title('Average Household Income by Store Location')
ax.set_xlabel('Store Location')
ax.set_ylabel('Average Household Income')
plt.xticks(rotation=0, ha='right')

ax.yaxis.set_major_formatter(mtick.StrMethodFormatter('£{x:,.0f}'))

ax.grid(axis='y', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```



```
In [385... income_by_store #Average household income by store location
```

```
Out[385... store_location
Dereham      41696.124626
Wymondham    41661.956126
Thetford     41268.514048
Fakenham     39941.384390
Norwich      39636.589373
Name: household_income, dtype: float64
```

Learning Task

Predict customer’s satisfaction score based on their demographics and shopping behaviour.

- Task type: Regression
- Target column: satisfaction


```
In [386... #Counts of values in satisfaction column from different DataFrames
print(df['satisfaction'].value_counts(dropna=False))
print(supermarket['satisfaction'].value_counts(dropna=False)) #Original df
```

```
satisfaction
Medium      1100
High         987
Very High   975
Low          961
Very Low    955
Name: count, dtype: int64

satisfaction
Medium      1105
High         991
Very High   981
Low          966
Very Low    957
Name: count, dtype: int64
```

```
In [387... # Mapping
satisfaction_map = {
    'Very Low': 1,
    'Low': 2,
    'Medium': 3,
    'High': 4,
    'Very High': 5}

# Apply mapping to satisfaction column (assuming it exists)
if 'satisfaction' in df.columns:
    df['satisfaction_score'] = df['satisfaction'].map(satisfaction_map)

# Build total basket size
item_col = [
    'fruit_and_veg_items', 'meat_items', 'cupboard_items',
    'frozen_items', 'toiletries_items', 'other_items']

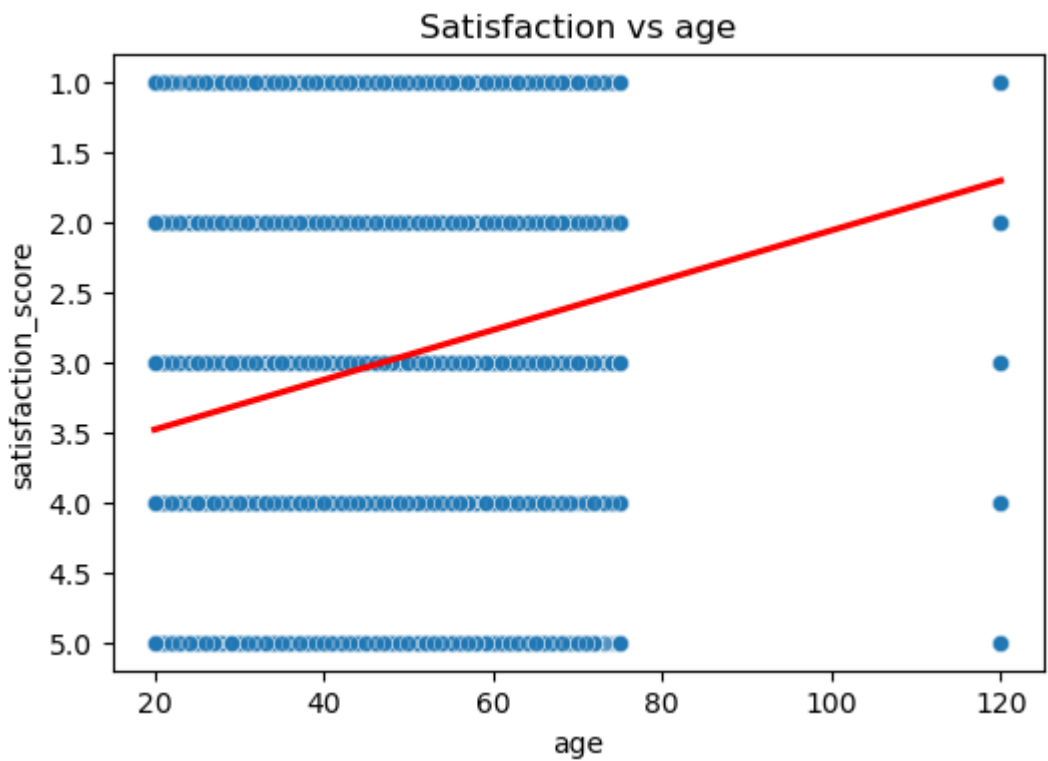
df['total_items'] = df[item_col].sum(axis=1)

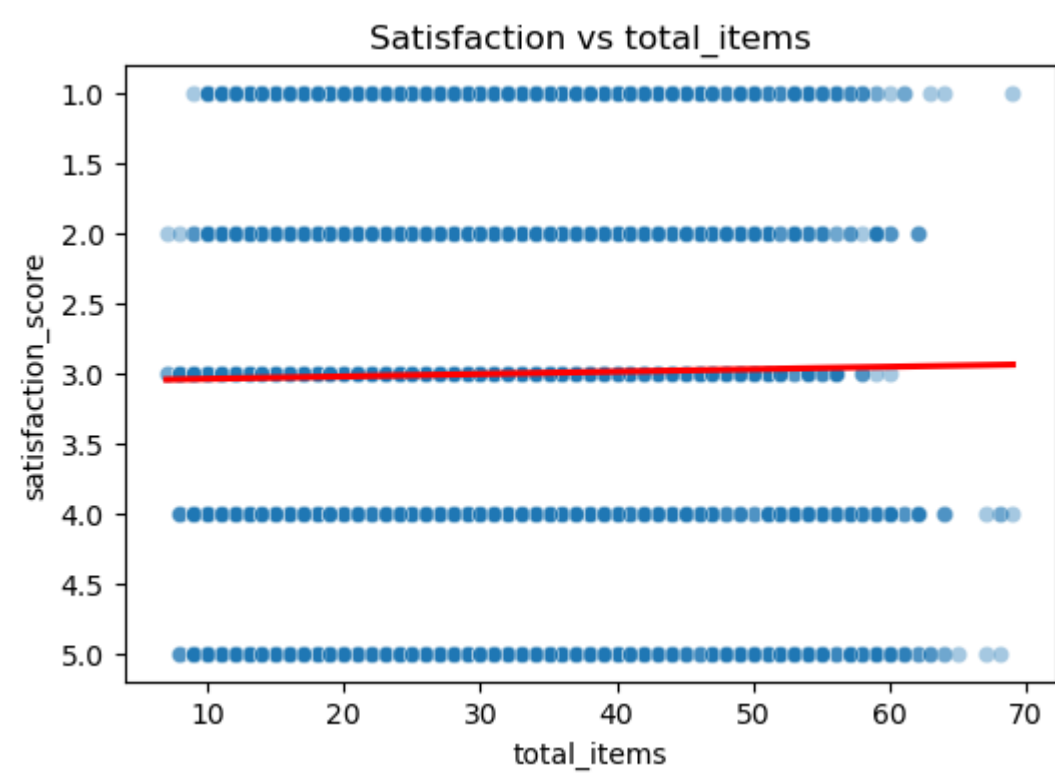
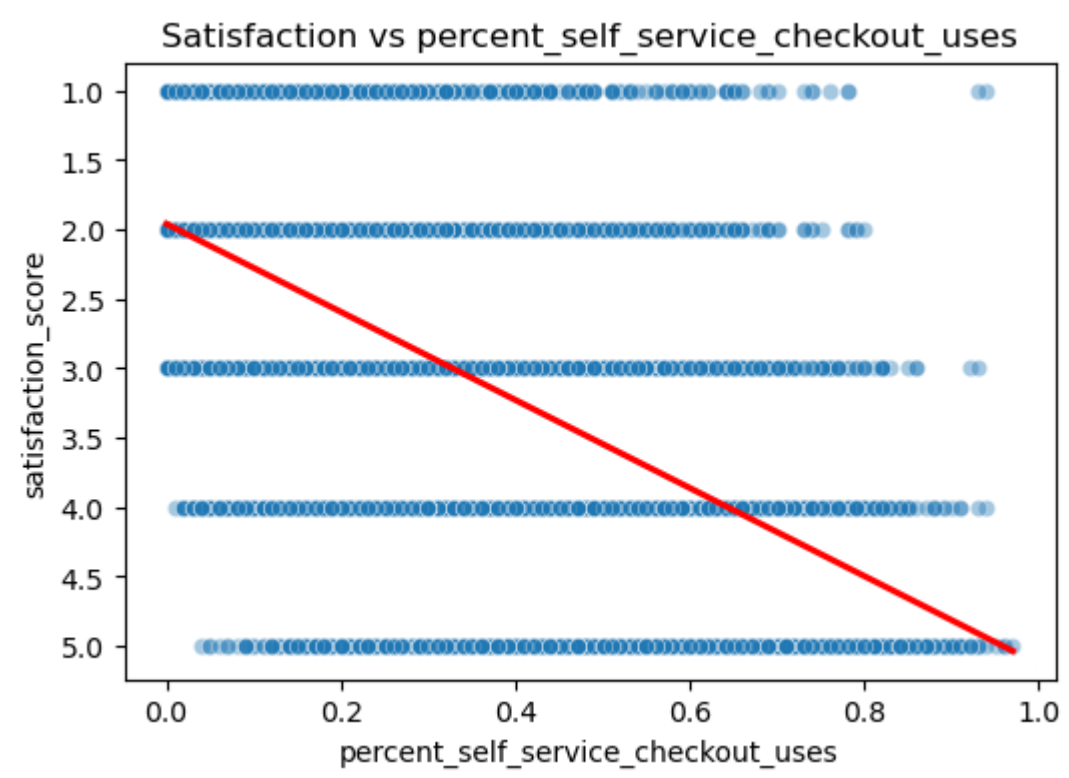
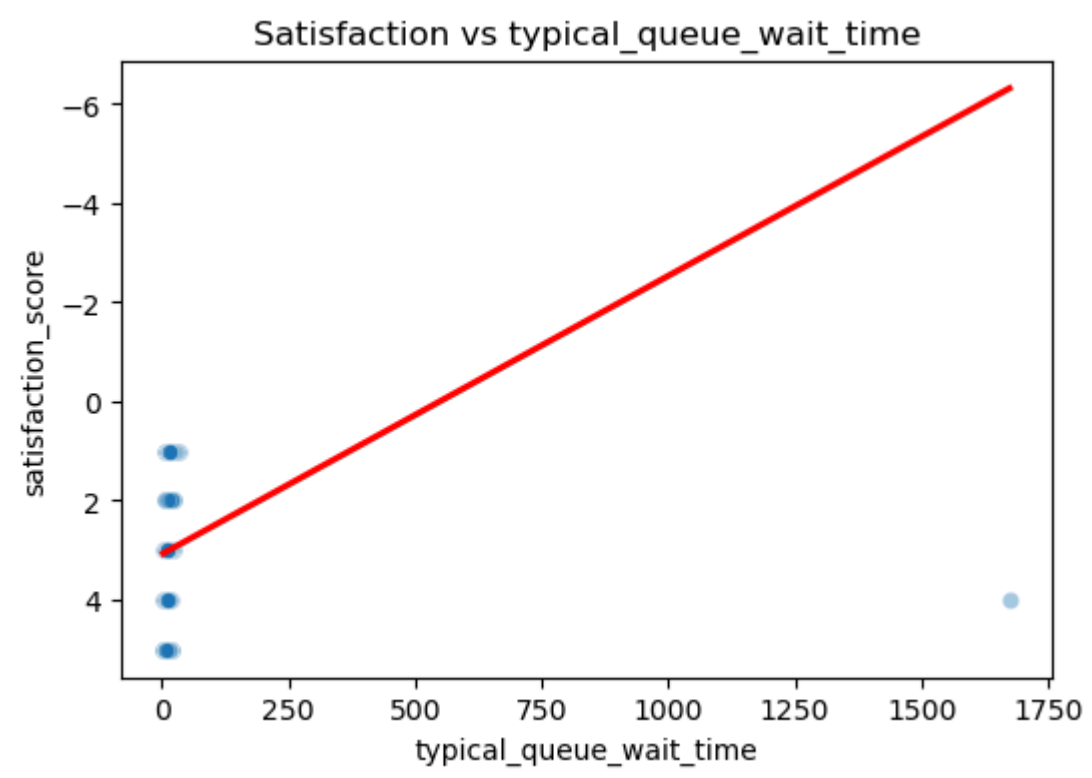
predictors_num = [
    'age', 'typical_queue_wait_time',
    'percent_self_service_checkout_uses',
    'total_items',
    'distance_to_store',
    'household_income',
    'average_time_in_store']

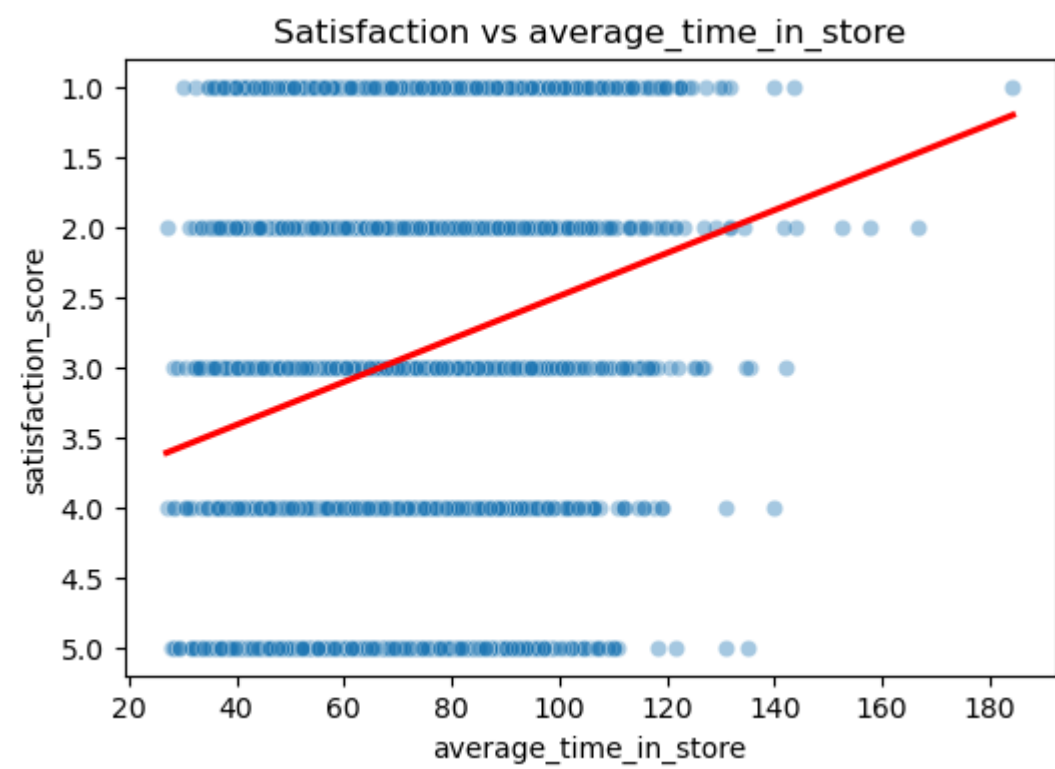
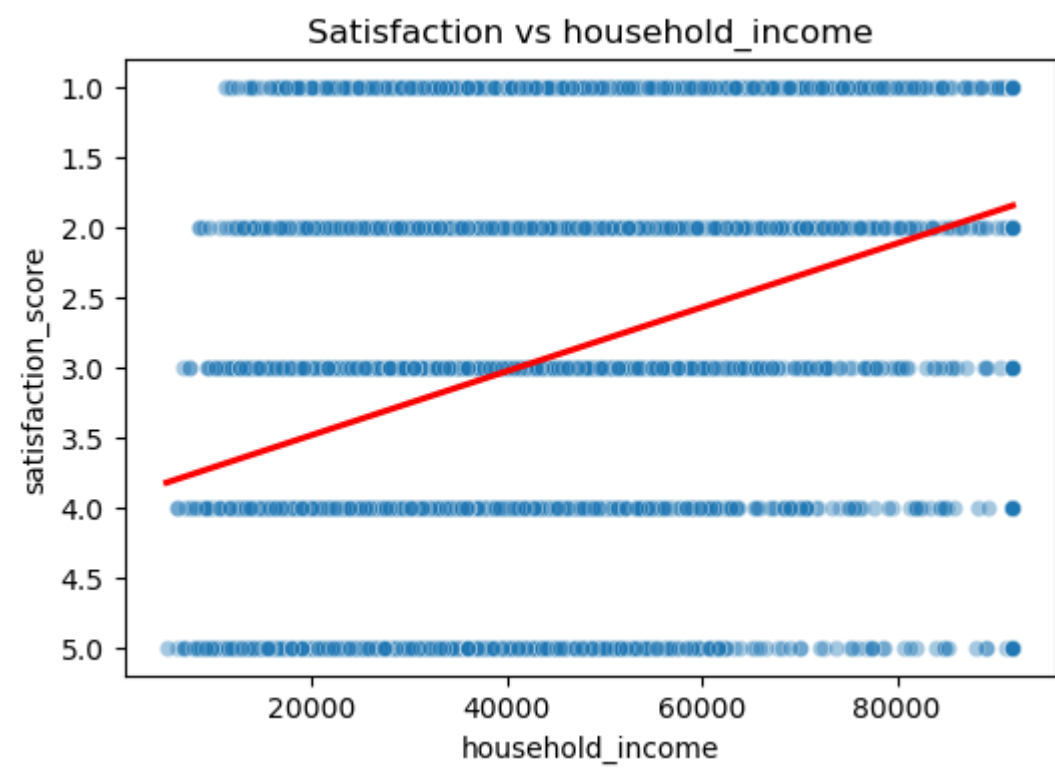
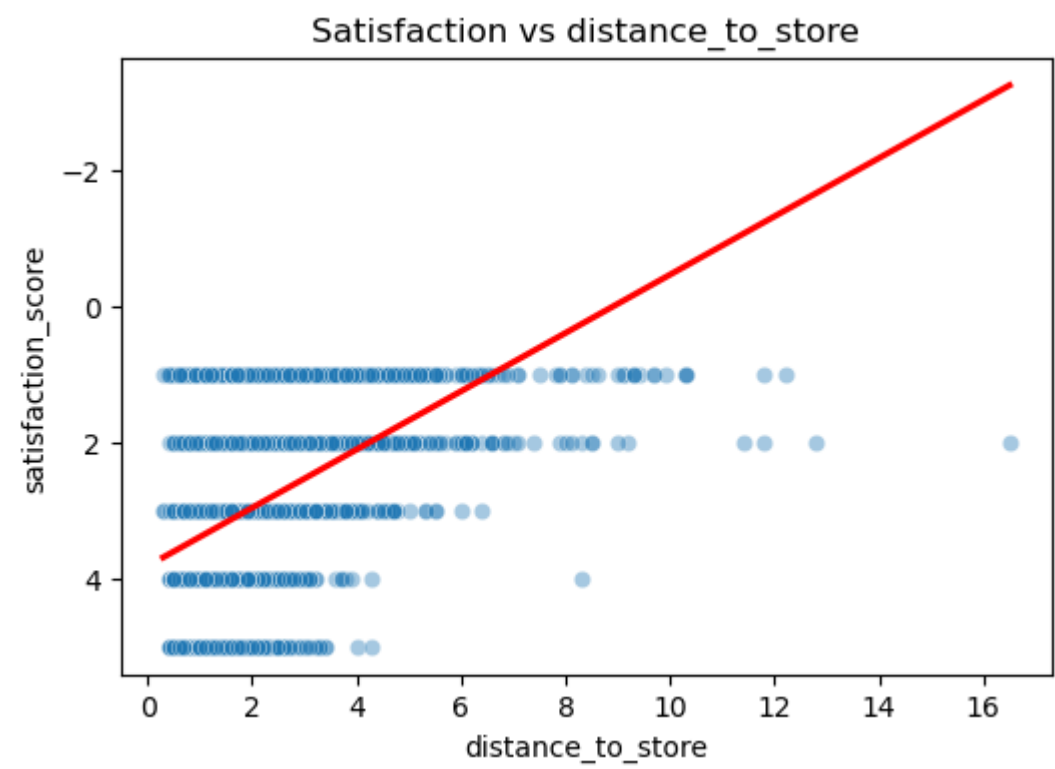
predictors_cat = ['typical_shopping_mission']
df.shape
```

Out[387... (4978, 23)

```
In [388... #Scatterplot for column in predictors_num (Figure 14)
for col in predictors_num:
    plt.figure(figsize=(6,4))
    sns.scatterplot(data=df, x=col, y='satisfaction_score', alpha=0.4)
    sns.regplot(data=df, x=col, y='satisfaction_score', scatter=False, color='red', ci=None)
    plt.title(f"Satisfaction vs {col}")
    plt.show()
```







```
In [389... #Keyword dictionary and themes in very low to low satisfaction comments
complaint_keywords = {
    "Navigation": ["find", "confusing", "layout", "aisles", "navigate", "hard"],
    "Staff": ["staff", "help", "service", "unhelpful", "rude", "uninterested"],
    "Checkout": ["queue", "approval", "card", "checkout", "till", "wait"],
    "Stock": ["stock", "out", "missing", "expiry", "availability"],
    "Cleanliness": ["messy", "dirty", "tired", "spillages", "clean"],
    "Parking": ["parking", "car park", "space", "cramped", "difficult to park", "traffic"],
    "Pricing": ["price", "expensive", "overpriced", "value", "deal", "offer", "pricing", "high"]}

lowsat_df = df[df["satisfaction"].isin(["Very Low", "Low"]).copy()
def get_themes(comment):
    themes = []
    text = str(comment).lower()
    for theme, words in complaint_keywords.items():
        if any(w in text for w in words):
            themes.append(theme)
```



```

    return themes
lowsat_df["themes"] = lowsat_df["Customer_Comment"].apply(get_themes)

from collections import Counter

all_themes = [t for sublist in lowsat_df["themes"] for t in sublist]
Counter(all_themes) #tallied frequencies with counter
```

```
Out[389... Counter({'Navigation': 510,
    'Checkout': 470,
    'Pricing': 469,
    'Stock': 419,
    'Staff': 340,
    'Parking': 193,
    'Cleanliness': 189})
```


Visualisations: All visualisations produced in the Jupyter Notebook, including those produced from both pre-transformation and post-transformation data.

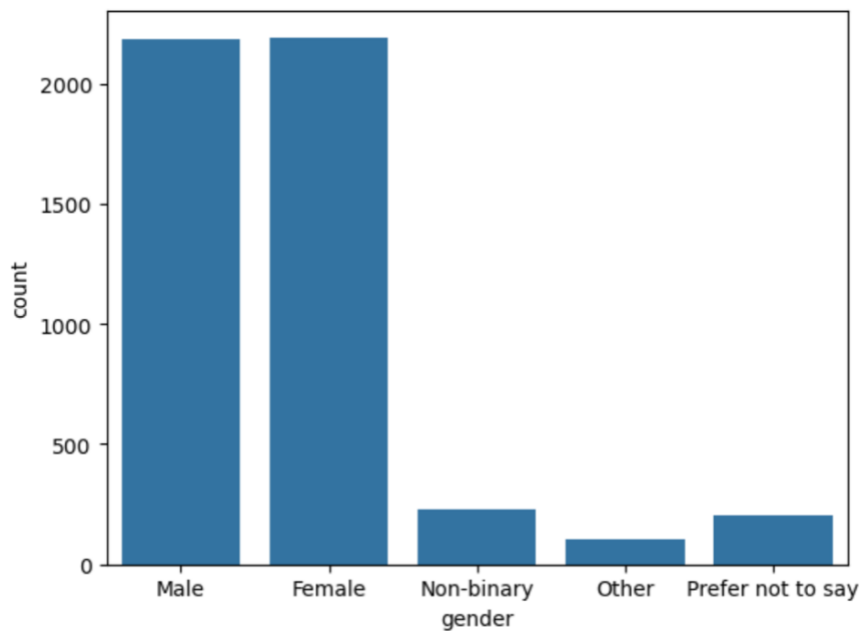


Figure 1

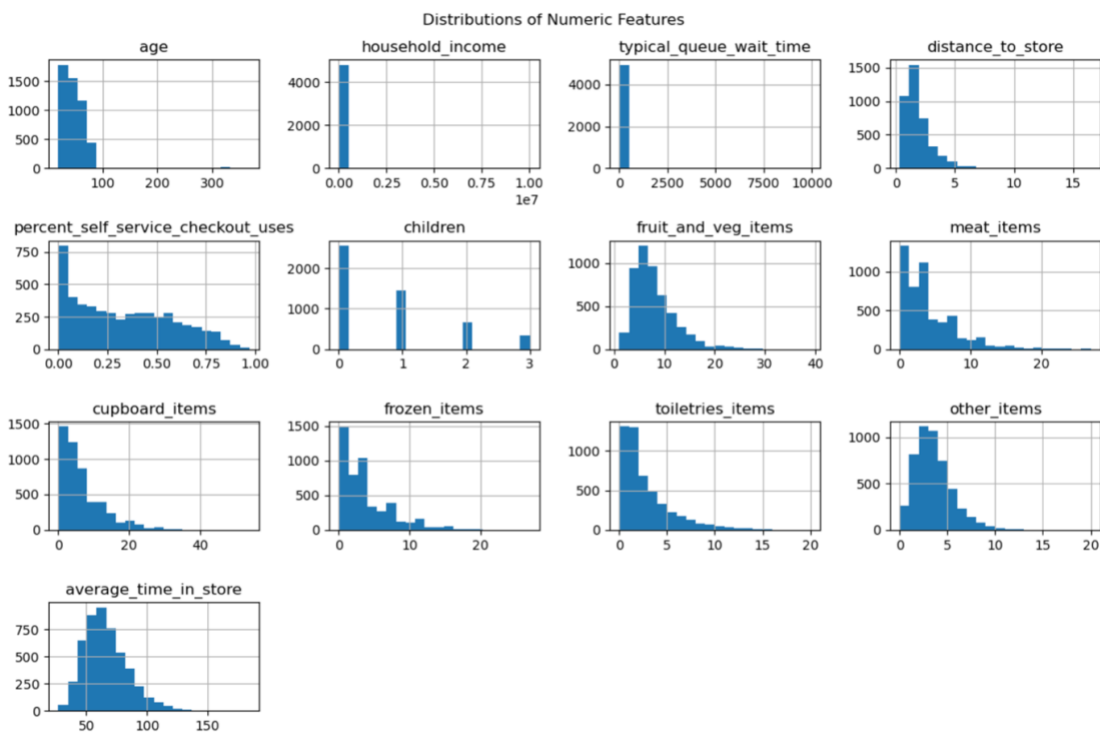


Figure 2

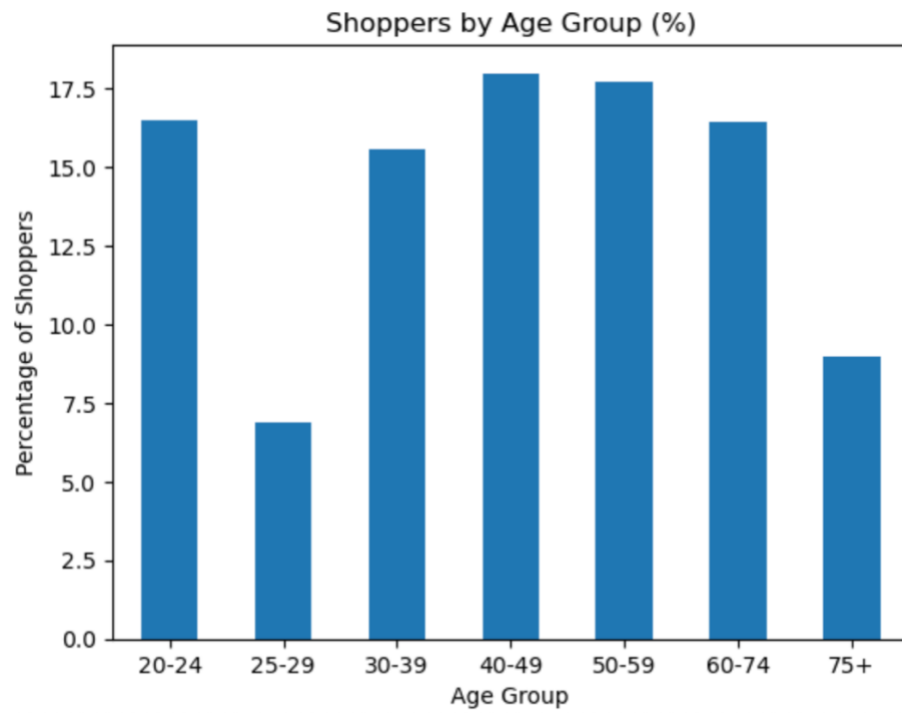


Figure 3

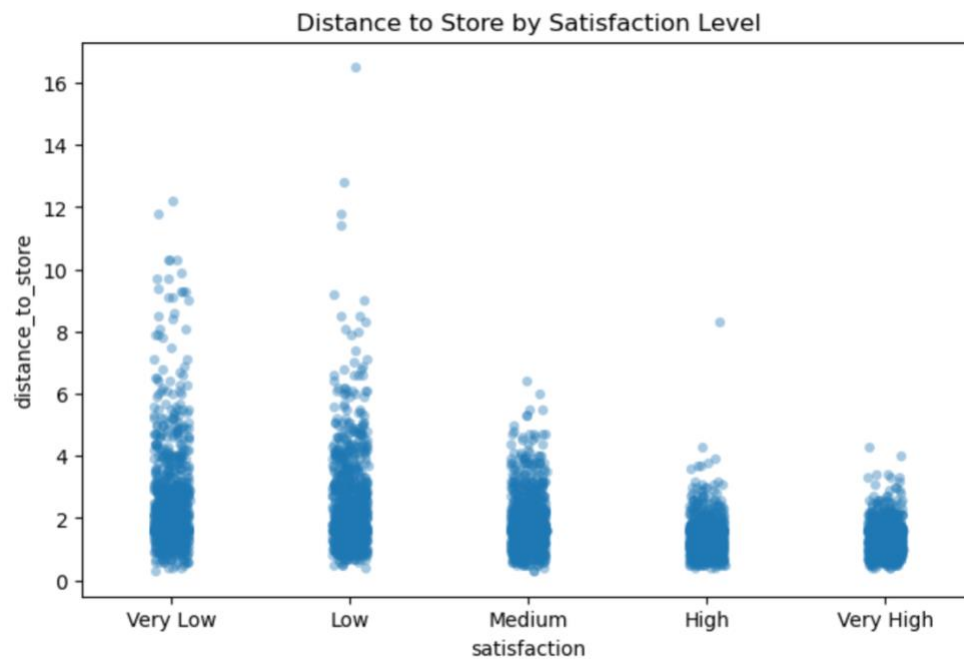


Figure 4

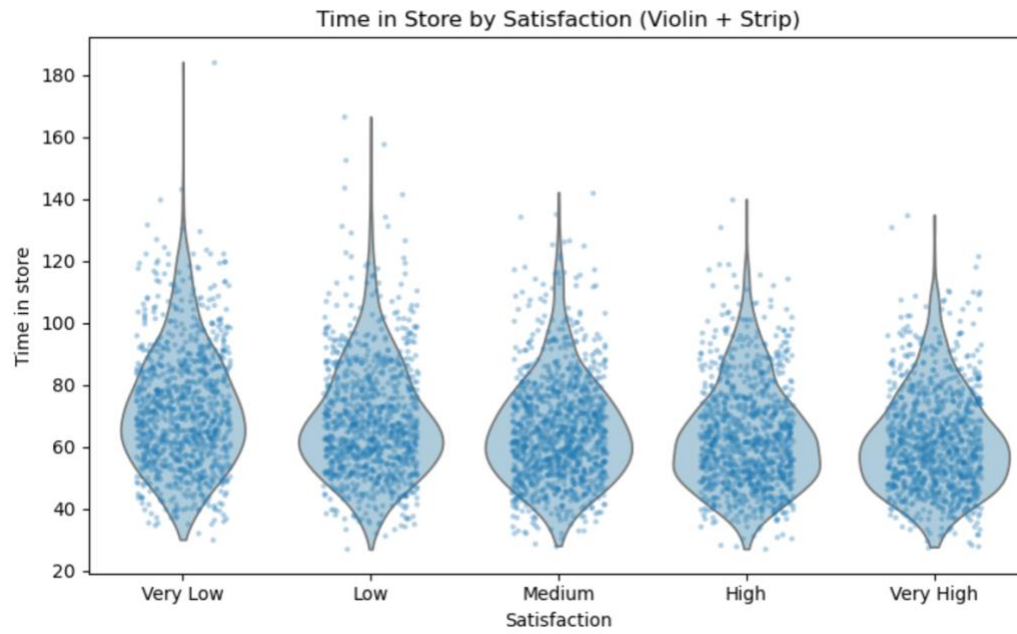


Figure 5

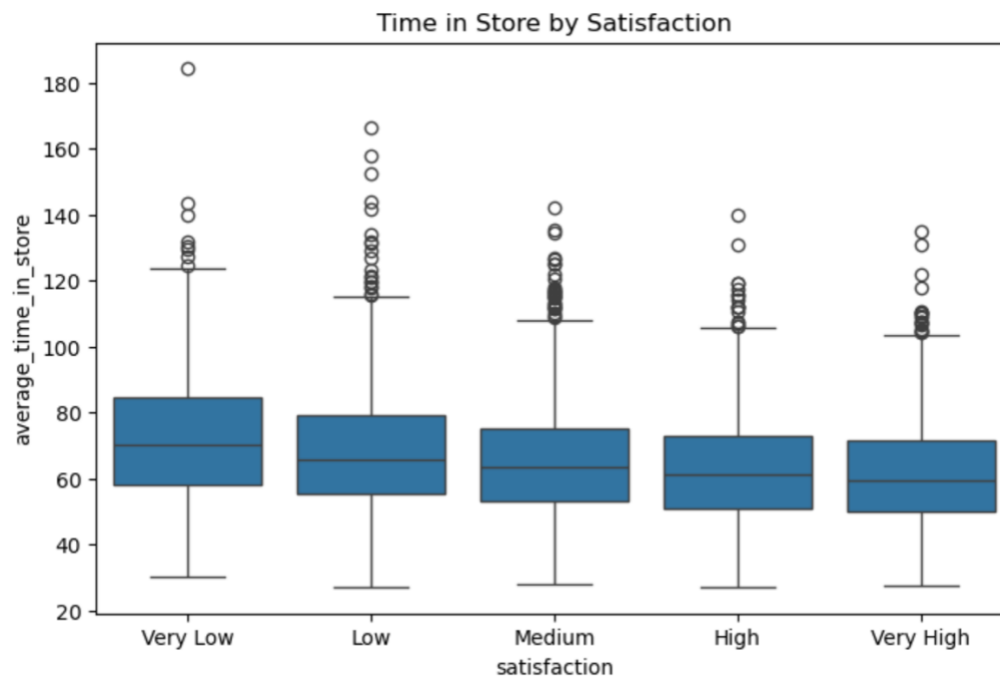


Figure 6

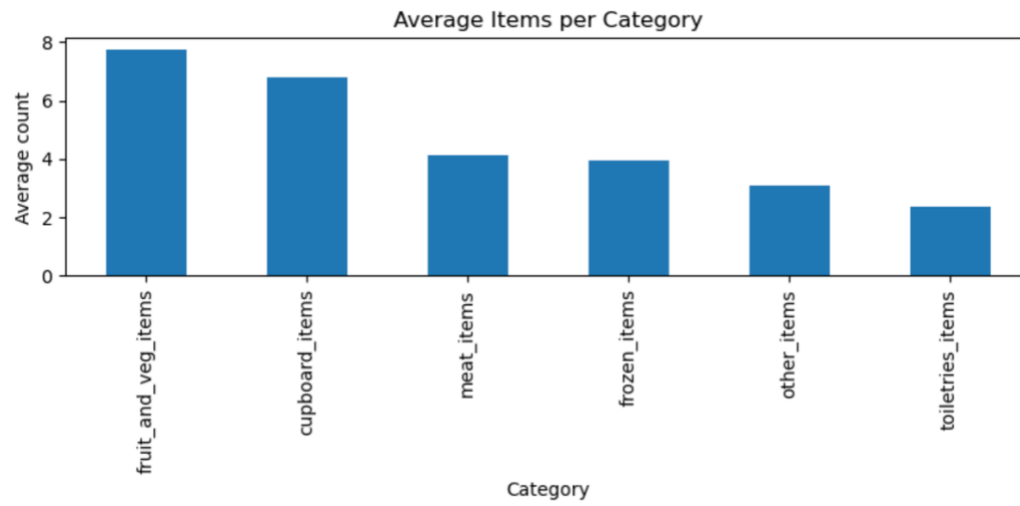


Figure 7

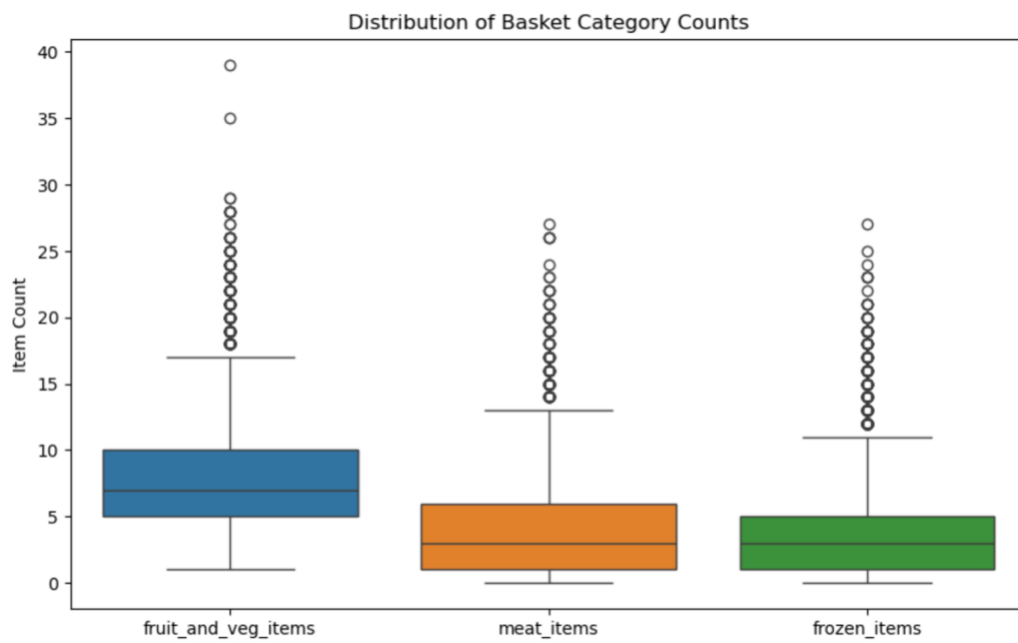


Figure 8

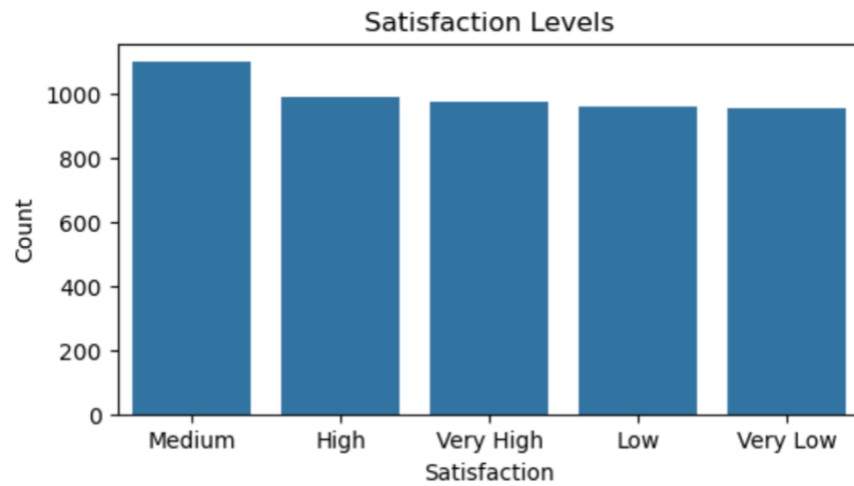


Figure 9



Figure 10

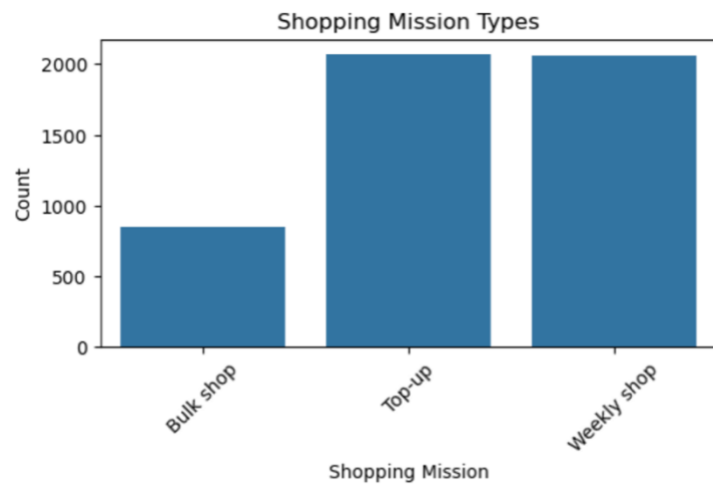


Figure 11

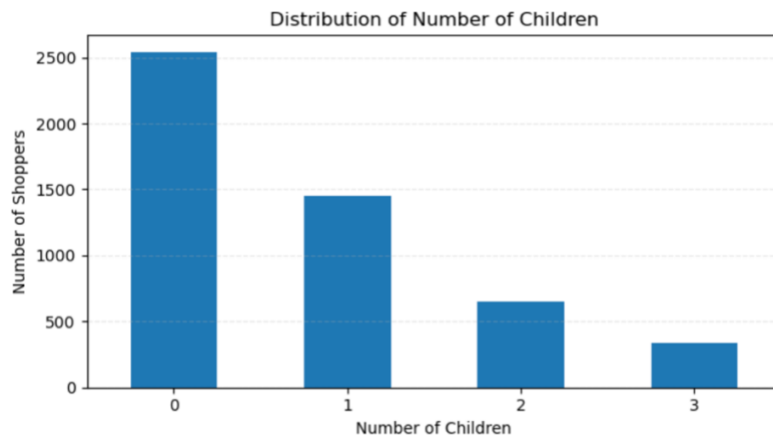


Figure 12

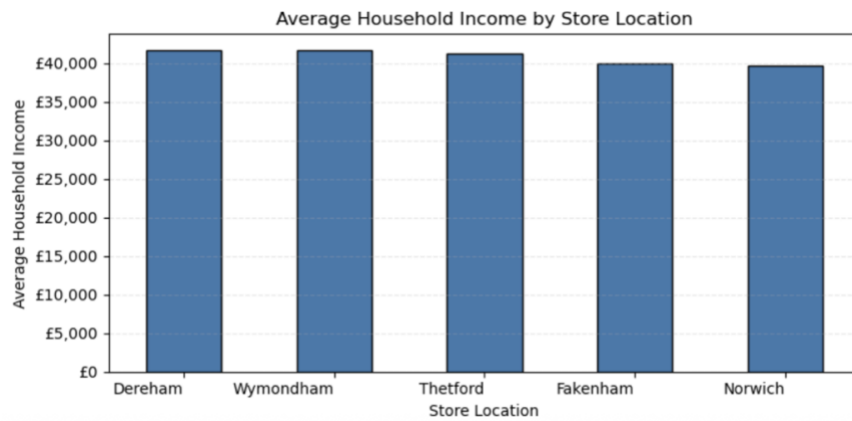
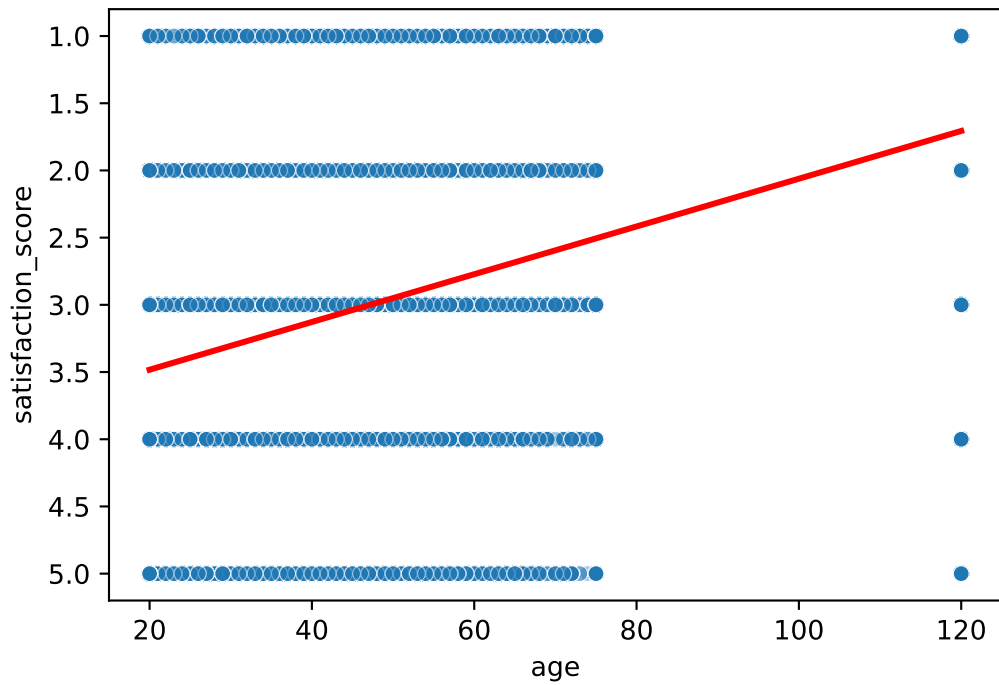
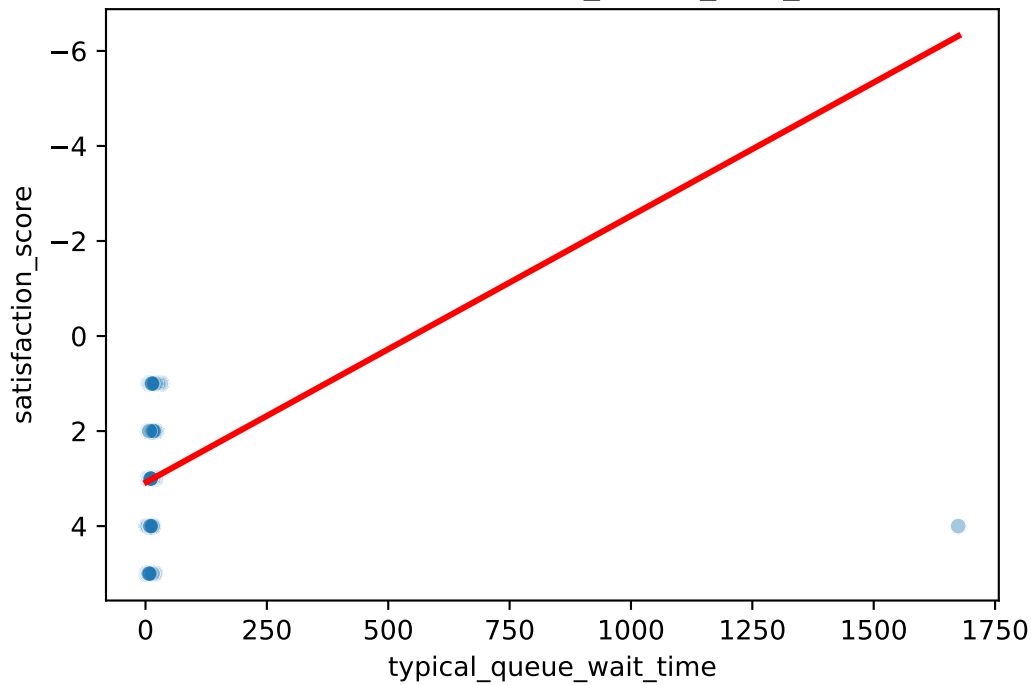


Figure 13

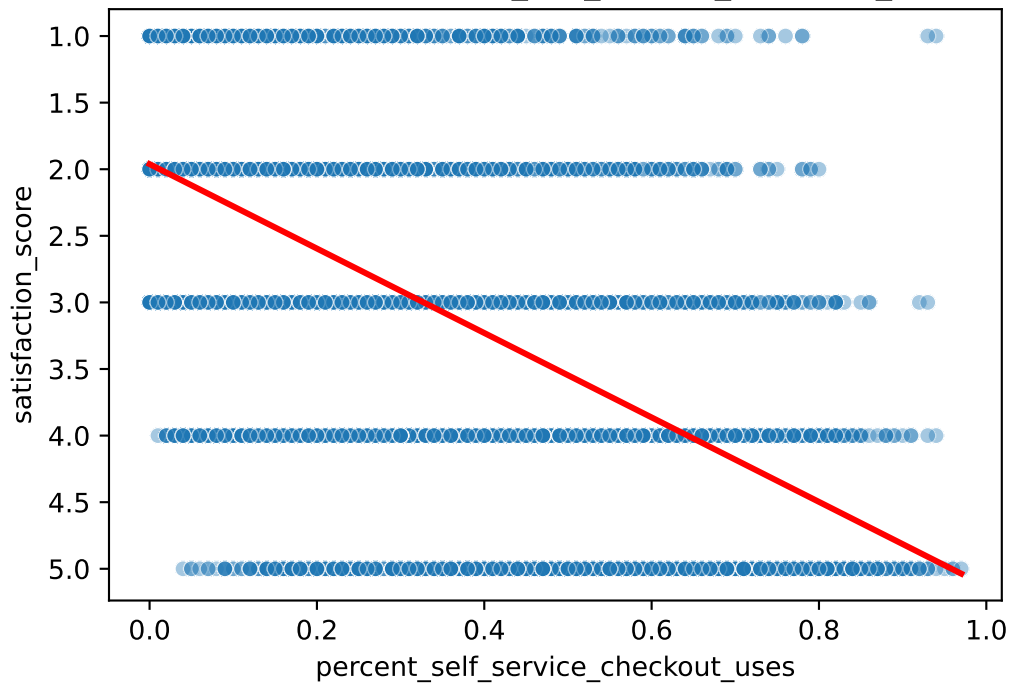
Satisfaction vs age



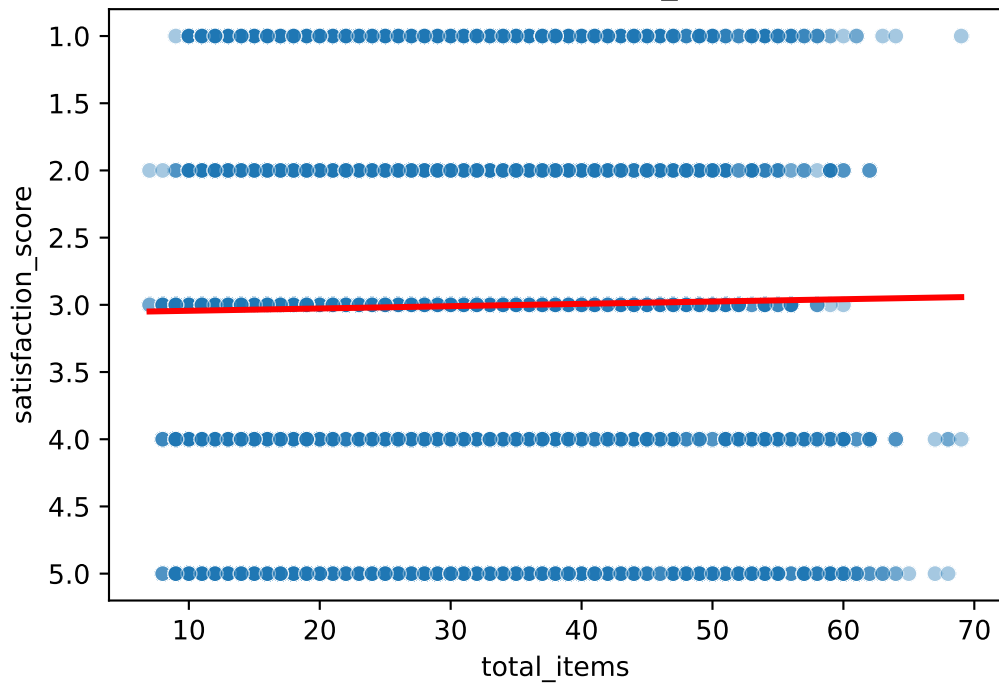
Satisfaction vs typical_queue_wait_time



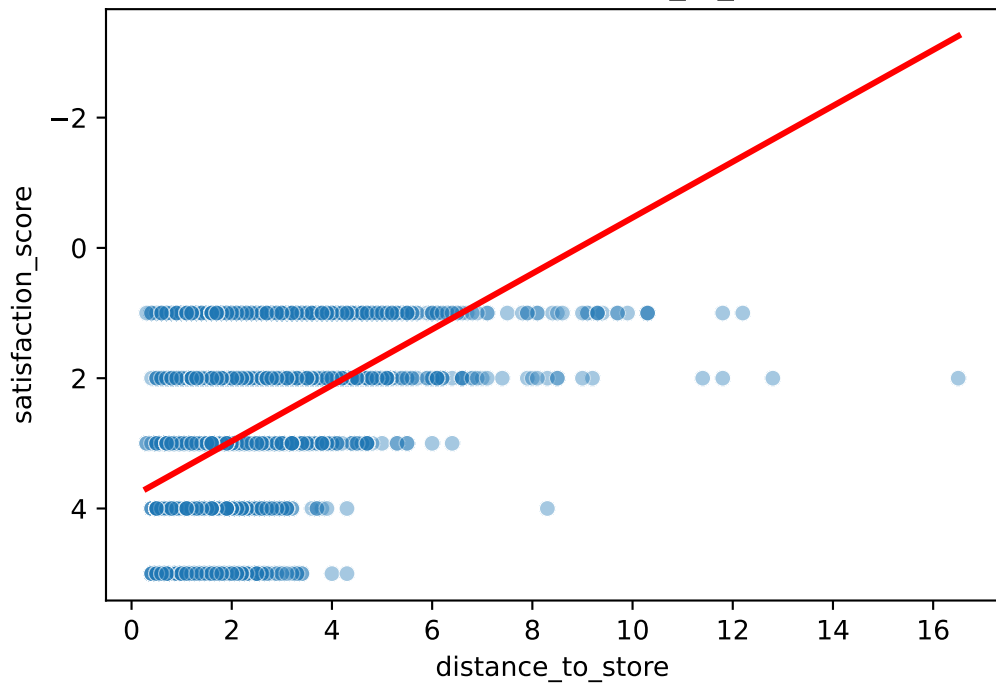
Satisfaction vs percent_self_service_checkout_uses



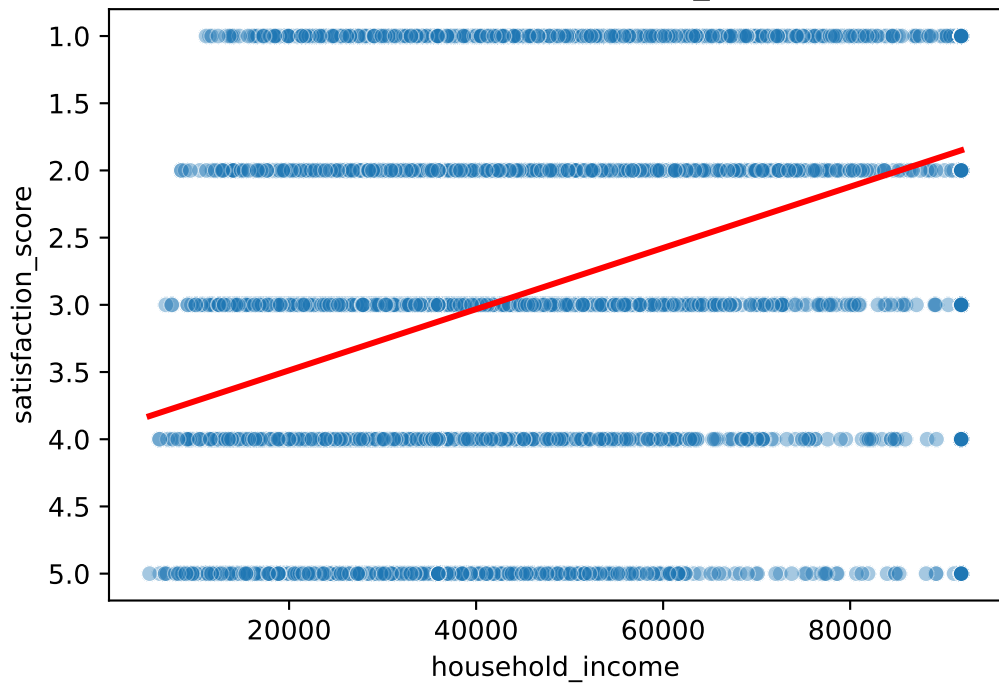
Satisfaction vs total_items



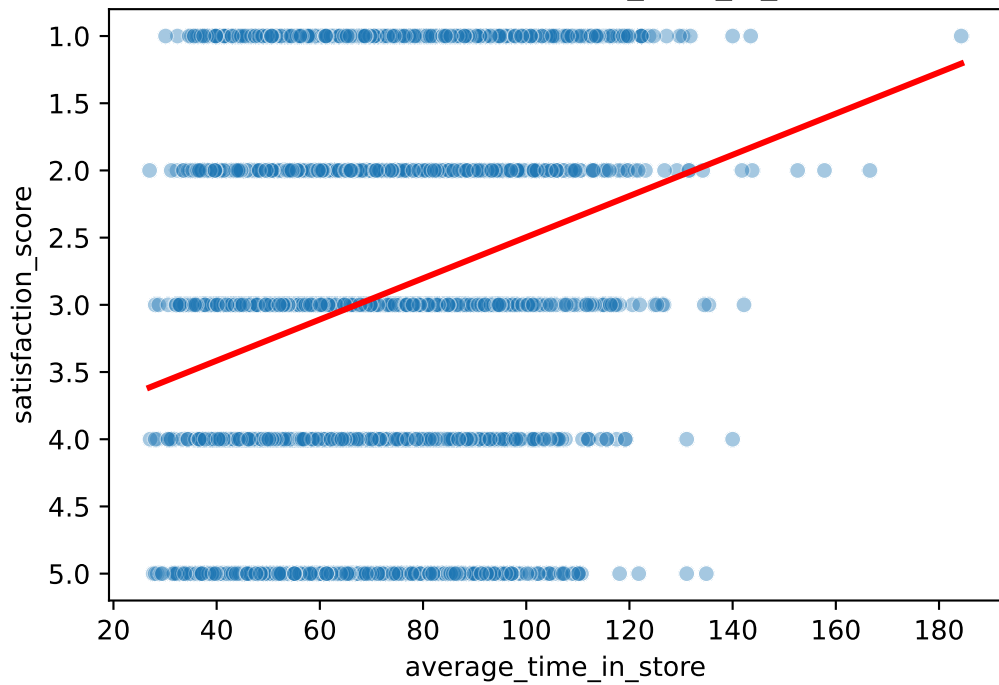
Satisfaction vs distance_to_store



Satisfaction vs household_income



Satisfaction vs average_time_in_store



Variable	Definition	Original Dtype	Cleaned Dtype	Accepts Nulls?	Missing Before	Notes
age	Respondent age in years	int64	int64	No	0	Outliers clipped during cleaning
household_income	Reported household income	float64	float64	Yes	188	Median-imputed
gender	Self-identified gender	object	category	Yes	81	Cleaned & recoded Fuzzy-matched & standardised
store_location	Store typically visited	object	category	No	0	
typical_queue_wait_time	Checkout queue waiting time (minutes)	float64	float64	Yes	48	Missing values imputed
distance_to_store	Distance from home to store	float64	float64	Yes	912	Heavy missingness; imputed
percent_self_service_checkout_uses	Proportion of trips using self-checkout	float64	float64	No	0	Normalised 0–1
children	Number of children	int64	int64	No	0	Cleaned
fruit_and_veg_items	Count of fruit & veg items	int64	int64	No	0	N/A
meat_items	Meat/fish items count	int64	int64	No	0	N/A
cupboard_items	Ambient/cupboard item count	int64	int64	No	0	N/A
frozen_items	Frozen item count	float64	int64	Yes	95	Converted to integer
toiletries_items	Toiletries/health items count	float64	int64	Yes	59	Converted to integer
other_items	Other miscellaneous items	int64	int64	No	0	N/A
has_loyalty_card	Loyalty card holder (Yes/No)	object	object	No	0	Cleaned categories
typical_visit_time	Morning/Afternoon/Evening	object	category	No	0	Standardised categories
typical_shopping_mission	Type of shop (Top-up, Weekly, Bulk, etc.)	object	category	No	0	Cleaned via fuzzy-match
average_time_in_store	Time spent in store	float64	float64	No	0	Cleaned
typical_payment_method	Usual payment method	object	category	Yes	199	Trimmed & harmonised
satisfaction	Satisfaction score	object	category	No	0	Ordinal categories
Customer_Comment	Free-text comment	object	object	Yes	1577	Text cleaned
brings_own_bag	Whether the customer brought their own bag (Yes/No)	object	Removed	No	0	Column removed during cleaning because it was not required for the analysis.
brand_loyalty	Self-reported brand loyalty level (Low/Medium/High)	object	Removed	No	0	Column removed during cleaning because it was not required for the analysis.
total_items	Total basket items (sum of item fields)	int64	int64	No	N/A	Derived feature
supermarket	Raw dataset loaded from SupermarketEvaluation.csv.	DataFrame	DataFrame	Yes	N/A	Source table prior to any cleaning or feature engineering.

tlog	Utility function that appends timestamped entries to travel_log.txt (task, why, tools, notes).	function	function	N/A	N/A	For workflow logging; does not touch data.
df	Working DataFrame derived from supermarket after dropping brings_own_bag, brand_loyalty.	DataFrame	DataFrame	Yes	N/A	Base table for cleaning & modeling. Column-level nulls tracked in main dictionary. Used by fix_city; values: Wymondham, Fakenham, Dereham, Norwich, Thetford.
valid_cities	Allowed city spellings for location standardisation. Fuzzy matcher (>0.75) that standardises store_location to the closest valid city.	list	list	N/A	N/A	
fix_city	Allowed shopping missions.	function	function	N/A	N/A	Uses difflib.get_close_matches. Values: Weekly shop, Top-up, Bulk shop.
valid_missions		list	list	N/A	N/A	
fix_mission	Fuzzy matcher (>0.75) that standardises typical_shopping_mission.	function	function	N/A	N/A	Uses difflib.get_close_matches. Columns: household_income, typical_queue_wait_time, distance_to_store, frozen_items, toiletries_items.
num_cols	Numeric columns imputed with each column's median.	list	list	N/A	N/A	
age_pct	% distribution of respondents by age bands	series	series	No	N/A	Depends on predefined bins & labels; represents shares by age band.
satisfaction_map	Mapping of satisfaction text ordinal (Very Low=1 Very High=5).	dict	dict	N/A	N/A	For encoding satisfaction into numeric scale. fruit_and_veg_items, meat_items, cupboard_items, frozen_items, toiletries_items, other_items.
item_col	Columns summed to create total_items. Derived feature: total basket size (row-wise sum of item_col).	list	list	N/A	N/A	
total_items		N/A	int64	No	N/A	Present in cleaned table: 4978 non-null rows.

							age, typical_queue_wait_time, percent_self_service_checkout_uses, total_items, distance_to_store, household_income, average_time_in_store.
predictors_num	Numeric predictors planned for modeling.	list	list	N/A	N/A		
predictors_cat	Categorical predictors planned for modeling.	list	list	N/A	N/A		
complaint_keywords	Theme keywords dictionary for tagging low-satisfaction comments.	dict	dict	N/A	N/A		typical_shopping_mission. Themes: Navigation, Staff, Checkout, Stock, Cleanliness, Parking, Pricing.
lowsat_df	Subset of df where satisfaction (Very Low, Low).	DataFrame	DataFrame	Yes	N/A		Used for complaint analysis and theme tagging.
get_themes	Function that returns list of matched theme labels per Customer_Comment.	function	function	N/A	N/A		Case-insensitive scan against complaint_keywords.
lowsat_df["themes"]	Column of detected themes for each low-satisfaction comment. Flattened list of all detected themes in lowsat_df.	series	series	No	N/A		May contain multiple themes per comment.
all_themes		list	list	No	N/A		Used for frequency analysis.
Counter(all_themes)	Frequency table of detected themes.	collections.Counter	collections.Counter	N/A	N/A		Tallies theme occurrences across low-satisfaction comments.
categorical_features	List of columns in supermarket whose dtype is object or category.	list	list	N/A	N/A		Used for identifying categorical vars before cleaning. Values are based on the <i>raw</i> dataset.
numerical_features	List of columns in supermarket whose dtype is numeric: int64 or float64.	list	list	N/A	N/A		Used to isolate numeric features for preprocessing. Based on the <i>raw</i> dataset; differs from cleaned df.

Example:

[TIMESTAMP]

Task completed:

Why I did it:

Tools/commands used (optional):

Notes:

[2026-02-05 21:15 UTC]

Task completed: Loaded CSV file and reviewed dataset structure

Why I did it: To begin familiarizing myself with the dataset and understand its shape, types, and data quality

Tools/commands used (optional): df.shape, df.info(), df.describe(), df.isnull().sum(), df.nunique(), df.head(), df.tail()

Notes: Initial scan of dataset; part of early draft assessment

[2026-02-09 13:51 UTC]

Task completed: Fixed typos and misspellings in store location and payment method columns

Why I did it: To ensure consistent categorical values before deeper analysis

Tools/commands used (optional): String cleaning, str.replace, mapping

Notes: Important for avoiding fragmented categories later in the analysis

[2026-02-10 00:36 UTC]

Task completed: Built and applied a replacement map for common typos

Why I did it: To automate future cleaning and increase reproducibility

Tools/commands used (optional): Dictionary mapping, replace()

Notes: Continued draft-level data cleaning

[2026-02-10 00:36 UTC]

Task completed: Identified columns containing NaNs and isolated rows with $\geq 50\%$ missing values

Why I did it: To assess data quality and decide how to handle incomplete entries

Tools/commands used (optional): df.isnull(), column subset filtering

Notes: Marked for later decisions on removal or imputation

[2026-02-13 00:16 UTC]

Task completed: Conducted preliminary EDA on shopper profile (demographics)

Why I did it: To get an early understanding of data distributions and potential segmentation

Tools/commands used (optional): Basic visualization and descriptive statistics

Notes: Draft analysis only – structure exploration stage

[2026-02-13 00:16 UTC]

Task completed: Updated data types for selected columns

Why I did it: To ensure correct interpretation of numerical and

boolean features

Tools/commands used (optional): astype() conversions

Notes: Updated: frozen_items(Int64), toiletries_items(Int64), has_loyalty_card(bool), brings_own_bag(bool)

[2026-02-13 00:16 UTC]

Task completed: Created age bins and explored shopping behaviors

Why I did it: To compare customer segments and understand behavior differences

Tools/commands used (optional): pd.cut, grouping, exploratory visuals

Notes: Draft exploration of self-service usage, time in store, shoppers with vs without children, income and gender by store location

[2026-02-18 12:19 UTC]

Task completed: Assessed shopping convenience and overall accessibility

Why I did it: To evaluate ease and customer experience quality

Tools/commands used (optional): Early scoring metrics, descriptive summarization

Notes: High-level exploratory review

[2026-02-18 12:19 UTC]

Task completed: Analyzed queue wait times and satisfaction metrics

Why I did it: To understand relationships between wait time, satisfaction, and shopping behavior

Tools/commands used (optional): Grouped means, correlations, distributions

Notes: Examined queue wait times → satisfaction, time in store by location and mission, loyalty card comparisons, self-checkout usage

[2026-02-18 12:19 UTC]

Task completed: Cleaned and refined other sections; checked for errors

Why I did it: To ensure consistency and reliability before deeper modeling

Tools/commands used (optional): Manual review, pandas cleanup

Notes: Part of ongoing draft refinement

[2026-02-23 14:29 UTC]

Task completed: Created a new notebook after experimenting with key exploratory data analysis (EDA) techniques using pandas, seaborn, and other relevant libraries to ensure familiarity with the analytical workflow. Imported starter libraries (Pandas, Seaborn, Numpy), load csv file, define a function to tasks into travel log

Why I did it: Needed to prepare for analysis

Tools/commands used (optional): Python, Pandas, Seaborn

Notes: Travel log will make recording tasks easier while staying in Jupyter Notebook

[2026-02-24 04:11 UTC]

Task completed: Performed data familiarization tasks such as

descriptive stats, info, dtypes, unique and NaN values
Why I did it: Needed to prepare for analysis and cleaning
Tools/commands used (optional): Python, Pandas
Notes: Check what dtype values need to be changed and/or what needs to be filled/dropped

[2026-02-24 17:12 UTC]

Task completed: Identified household income outliers using the IQR method and extracted categorical and numerical features
Why I did it: I needed to detect extreme income values that could impact analysis and to organize variables for structured EDA
Tools/commands used (optional): quantile(), IQR calculation, boolean filtering, select_dtypes()
Notes: IQR snippet was generated with ChatGPT; feature extraction approach came from a YouTube example. Verified results with .head() and printed feature lists.

[2026-02-24 17:14 UTC]

Task completed: Extracted categorical and numerical features and generated histograms for numerical variables
Why I did it: I needed to separate variable types for EDA and visualize distributions to understand data spread and potential skewness
Tools/commands used (optional): select_dtypes(), hist(), matplotlib, seaborn
Notes: Feature extraction and histogram approach were adapted from a YouTube tutorial. Verified feature lists using print statements and reviewed distribution plots for initial insights.

[2026-02-24 20:34 UTC]

Task completed: Created a cleaned working dataframe by dropping low-value columns, standardizing city names via fuzzy matching, and correcting payment method and shopping mission typos
Why I did it: I needed a consistent dataset with normalized categories to ensure reliable EDA and modeling
Tools/commands used (optional): pandas drop(columns), difflib.get_close_matches (fuzzy matching), .apply(), .astype('string'), .str.strip(), .replace(), .value_counts()
Notes: Established new df from supermarket after removing 'brings_own_bag' and 'brand_loyalty'. Standardized 'store_location' and 'typical_shopping_mission' with fuzzy matching against validated lists (cutoff=0.75). Manually fixed frequent typos in 'typical_payment_method'. Verified category cleanup using value_counts().

[2026-02-24 23:41 UTC]

Task completed: Clipped age and household income values to defined acceptable ranges
Why I did it: I needed to constrain extreme age and income values to

reduce the influence of outliers and ensure cleaner distributions for analysis

Tools/commands used (optional): pandas .clip(), describe(), IQR-based bounds

Notes: Set age limits to 18–120 and clipped values accordingly.

Applied IQR-derived lower and upper bounds to household_income to cap extreme values. Verified results using .describe().

[2026-02-27 00:26 UTC]

Task completed: Filtered unrealistic queue wait times, imputed missing values, and standardized data types (including ordered categoricals)

Why I did it: To ensure valid ranges, reduce bias from missing data, and create a dataset with consistent and meaningful dtypes for EDA and modeling

Tools/commands used (optional): pandas boolean filtering, describe(), fillna(), median(), astype(), pd.CategoricalDtype

Notes: Applied queue wait time cap: typical_queue_wait_time.Filled

text NA: Customer_Comment to (empty string). Filled categorical NA:

typical_payment_method and gender to Unknown. Imputed numeric NA with column medians for

['household_income', 'typical_queue_wait_time', 'distance_to_store', 'frozen_items', 'toiletries_items']. Converted to int: frozen_items, toiletries_items. Converted to categorical: typical_payment_method, gender, store_location, typical_shopping_mission. Created ordered categories: satisfaction and typical_visit_time.

[2026-02-28 11:38 UTC]

Task completed: Find shopper age-group percentages and created multiple EDA visualizations (bars, strip, violin+strip, boxplots) for satisfaction, distance, time in store, and basket composition

Why I did it: To quantify the distribution of shoppers by age segment and visualize key relationships/patterns that inform satisfaction drivers and shopping behavior

Tools/commands used (optional): pandas pd.cut(), value_counts(normalize=True), sort_index(); matplotlib.pyplot; seaborn (striplot, violinplot, boxplot, countplot); pandas .mean()

Notes: Computed age-group percentages and generated multiple EDA visuals, including age-group bar charts, distance vs. satisfaction strip plots, violin/strip and boxplots for time-in-store, average item-category bars, basket-composition boxplots, and count plots for satisfaction and shopping mission.

[2026-02-28 13:25 UTC]

Task completed: Compared satisfaction distributions across raw and cleaned data, plotted satisfaction relationships, and extracted complaint themes

Why I did it: To validate consistency after cleaning, create predictors for modeling, visualize satisfaction drivers, and surface common issues from low/medium satisfaction comments

Tools/commands used (optional): pandas value_counts(), sum(axis=1);

feature mapping (satisfaction→numeric); seaborn scatterplot/regplot;
custom keyword matching; collections.Counter

Notes: Compared df vs supermarket satisfaction counts (dropna=False).
Mapped satisfaction to numeric scale {Very Low:1 ... Very High:5}. Built
total_items from item columns and defined predictor sets (numeric +
categorical). Plotted satisfaction_num vs each numeric predictor with
scatter + trend line to inspect linear patterns. Created keyword
dictionary and tagged themes in low/medium satisfaction comments;
tallied frequencies with Counter.

[2026-03-01 22:35 UTC]

Task completed: Created additional exploratory visualizations
(satisfaction overall, satisfaction by store location, distribution of
number of children, and average household income by store)

Why I did it: To perform EDA and understand customer satisfaction
patterns and demographics across locations

Tools/commands used (optional): Python, Pandas, Matplotlib, Seaborn

Notes: Used colorblind-safe palette; added bar labels for counts;
formatted income axis as with separators; sorted categories/indexes;
applied tight_layout and set readable figure sizes